

The image shows a well-lit industrial networking lab. In the foreground, there are wooden-topped workbenches with black frames and multiple drawers. A black ergonomic chair is positioned in front of one of the benches. In the background, there are several racks filled with industrial networking equipment, including switches and routers, with various cables connected. A projector is mounted on the ceiling, and the room has a clean, professional appearance.

INDUSTRIAL NETWORKING

MENG 4010

CONTENTS

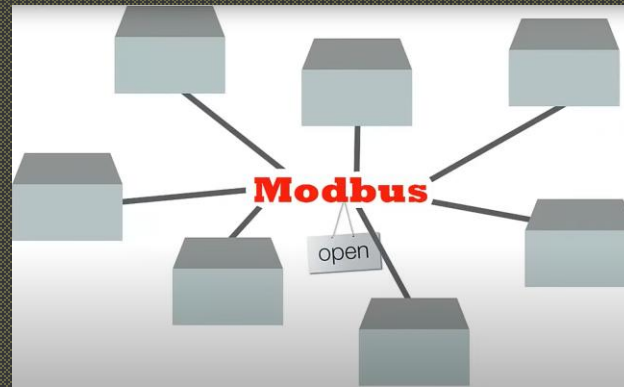
7A. Modbus Protocol

- 7A.1. Introduction to Modbus
- 7A.2 Modbus Fundamentals
- 7A.3 Serial Communication (RTU and ASCII)
- 7A.4 Modbus TCP Communication
- 7A.5 Modbus Function Codes and Data Models
- 7A.6 Modbus Security
- 7A.7 Modbus in Siemens and Allen-Bradley PLCs

7A.1 Industrial Communication and Modbus



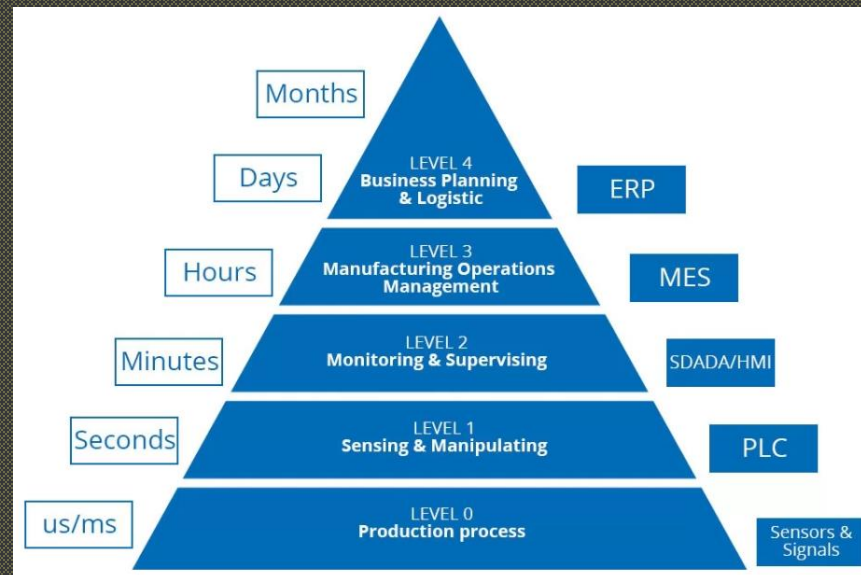
- In modern industrial automation systems, communication between devices in a LAN is fundamental.
- Industrial plants typically consist of multiple components - sensors, actuators, Programmable Logic Controllers (PLCs), Human-Machine Interfaces (HMIs), and Supervisory Control and Data Acquisition (SCADA) systems - that must exchange data to monitor and control processes effectively.
- To enable this communication, standardized protocols are required so that devices from different manufacturers can understand one another. One of the most widely adopted and time-tested protocols for this purpose is Modbus.
- Originally developed by Modicon (now part of Schneider Electric) in 1979, Modbus has become a cornerstone of industrial networking due to its simplicity, reliability, and open availability.
- Despite being over four decades old, it remains in extensive use today in manufacturing, energy management, building automation, and process control applications.



7A.1 Industrial Communication and Modbus

Automation systems operate on a hierarchy of communication levels. At the lowest level, field devices such as sensors and actuators generate raw data or perform actions. These devices communicate with controllers (PLCs, RTUs) that process signals and make decisions. At higher levels, HMIs and SCADA systems collect, display, and log process information for operators or engineers.

- Upward communication allows measurement data and system status to be transmitted from field devices to control and monitoring systems.
- Downward communication enables commands or setpoints from controllers to reach actuators and other output devices.
- Without a standard protocol, devices from different vendors might use incompatible communication formats, leading to data loss or system malfunction. This necessity for interoperability led to the development of open standards like Modbus, Profibus, and Ethernet/IP.



<https://excelpro.ca/en/news/the-automation-pyramid-isa-95>

7A.1 Industrial Communication and Modbus

Modbus is an application-layer protocol used for transmitting information over serial lines or Ethernet connections between electronic devices. It defines how messages are structured, addressed, and interpreted, ensuring that each device on a network can communicate successfully.

Modbus follows a client/server (master/slave) architecture:

- The client (or master) initiates communication by sending a request.
- The server (or slave) responds with the requested data or confirmation of an action.

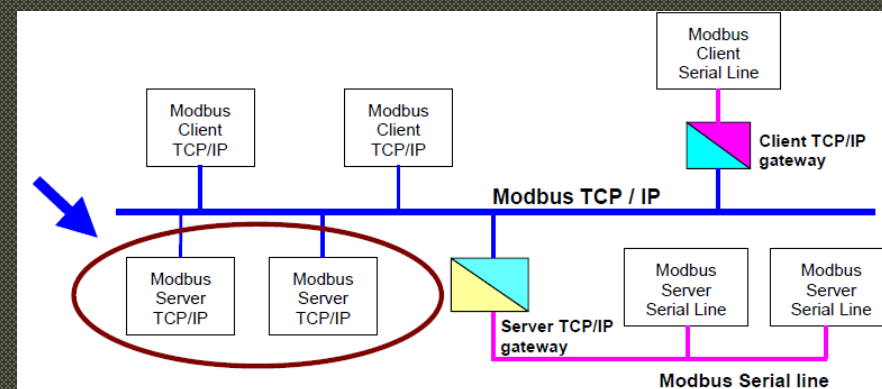
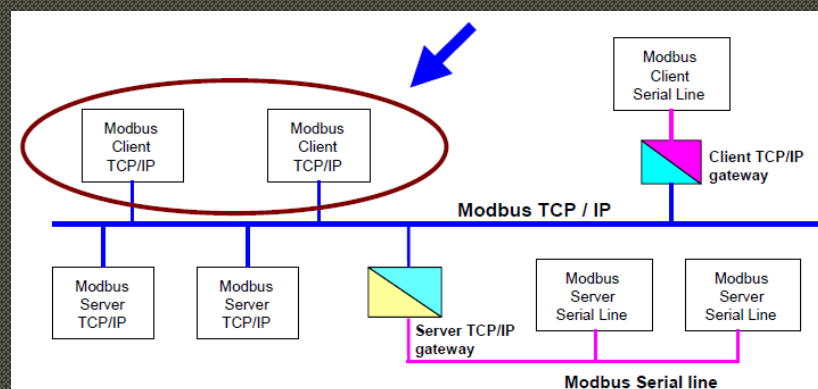
A Modbus message can instruct a device to:

- Read process variables (e.g., temperature, pressure, voltage)
- Write control commands (e.g., start a motor, open a valve)
- Monitor and log data for supervisory control

Because of its open and simple design, Modbus became one of the first truly interoperable protocols in industrial control. Its specification is public, meaning any manufacturer can implement it without licensing fees.

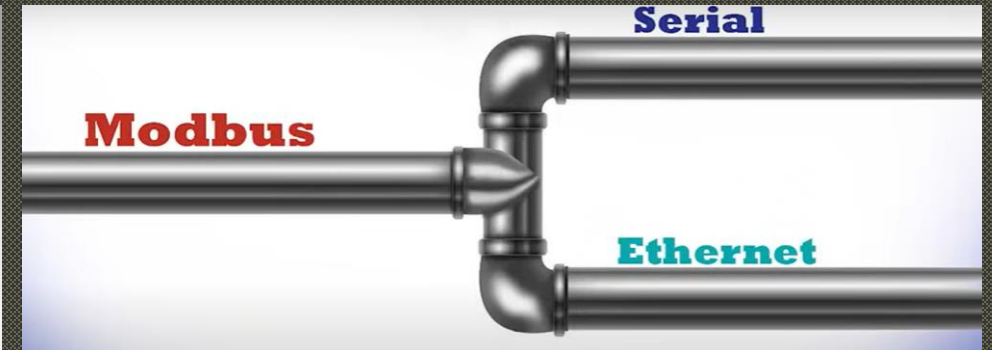
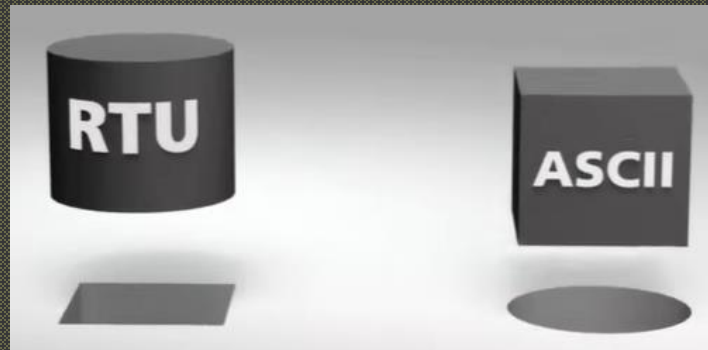
3 Layers used for Modbus Serial

APPLICATION	Modbus Protocol
PRESENTATION	Not used
SESSION	Not used
TRANSPORT	Not used
NETWORK	Not used
DATALINK	Master / Slave, Transmission RTU / ASCII
PHYSICAL	RS485, RS422, Fiber Optic, Radio, Cellular, ...



7A.1 Industrial Communication and Modbus

The original Modbus protocol was designed for serial communication using RS-232 interfaces. Later, as industrial networks evolved, Modbus was adapted for RS-485 (multi-drop networks) and eventually for Ethernet. The major variants are:



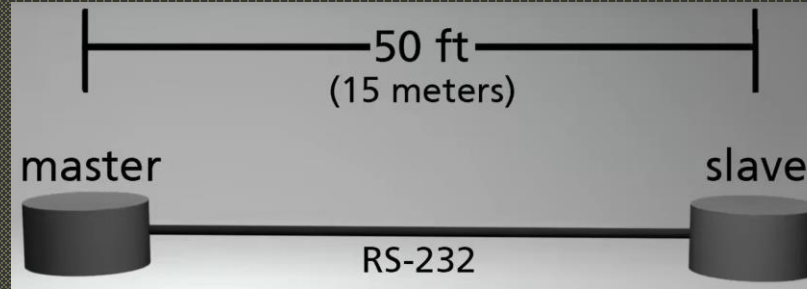
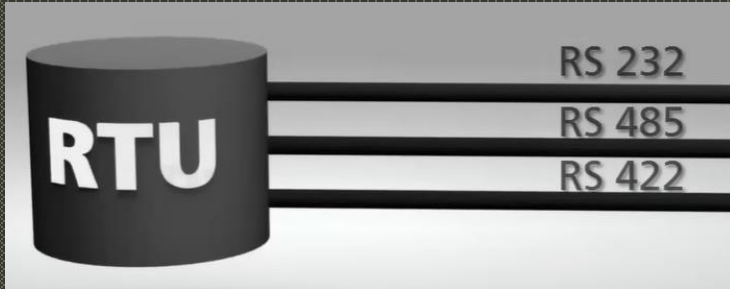
Modbus RTU and Modbus ASCII are incompatible

<https://www.prosoft-technology.com/Landing-Pages/Protocol/Modbus-and-Modbus-TCP-Protocol>

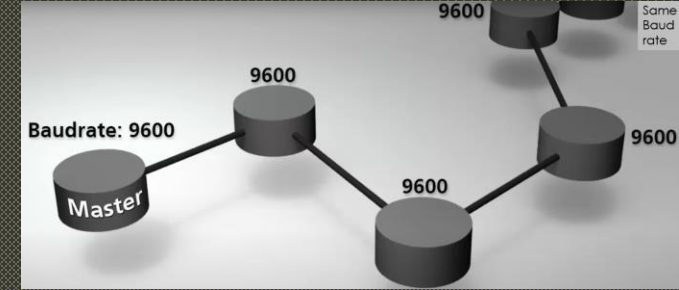
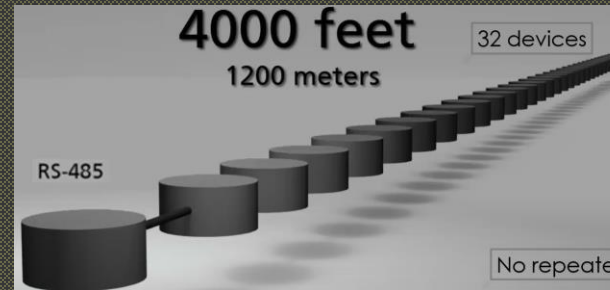
- Modbus RTU (Remote Terminal Unit) remains common in embedded and field devices due to its simplicity and low communication overhead.
- Modbus TCP/IP, introduced in the 1990s, integrates the Modbus protocol with standard Ethernet and TCP/IP technology, allowing data exchange across local and wide area networks.

Variant	Communication Medium	Transmission Mode	Typical Use
Modbus RTU	Serial (RS-485/RS-232)	Binary	Industrial devices, PLCs, energy meters
Modbus ASCII	Serial (RS-232/RS-485)	ASCII characters	Legacy systems, simple devices
Modbus TCP/IP	Ethernet	TCP/IP packets	Modern automation, SCADA, IIoT

7A.1 Industrial Communication and Modbus



<https://www.prosoft-technology.com/Landing-Pages/Protocol/Modbus-and-Modbus-TCP-Protocol>



Standard	Communication Topology	Max Distance*	Max Number of Devices	Typical Speed
RS-232	Point-to-Point (One-to-One)	15 meters	2	19.2 kbps
RS-422	Multi-Drop (One Master, Multiple Slaves)	1,200 meters	10	100 kbps - 10 Mbps
RS-485	Multi-Drop (Multi-Point Bus)	1,200 meters	32 per segment	100 kbps - 10 Mbps

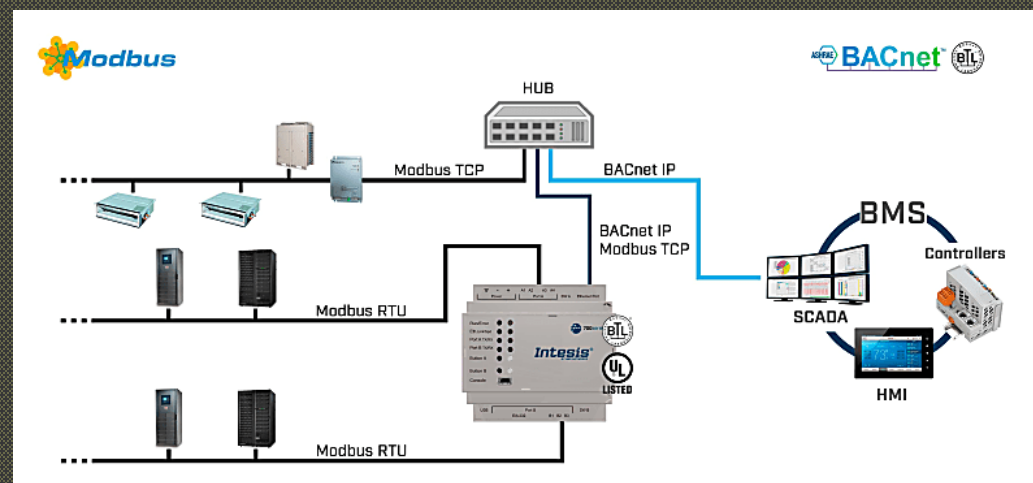
*Note: Maximum distance is dependent on data speed; lower speeds allow for longer cable runs.

7A.1 Industrial Communication and Modbus

Before standardization, industrial systems relied on proprietary communication methods. Devices from different manufacturers could not easily share data, forcing users to depend on single-vendor solutions. Modbus changed this landscape by providing:

- Open access: Any vendor can implement Modbus without licensing costs.
- Interoperability: Devices from multiple manufacturers can communicate seamlessly.
- Scalability: The same logical data model can be applied from small systems to plant-wide networks.
- Ease of implementation: Straightforward frame structure and function codes.

These advantages made Modbus the “universal language” for industrial devices. Even with the emergence of more sophisticated protocols (e.g., EtherNet/IP, Profinet, OPC UA), Modbus remains widely used due to its simplicity, legacy compatibility, and broad support.



7A.1 Industrial Communication and Modbus

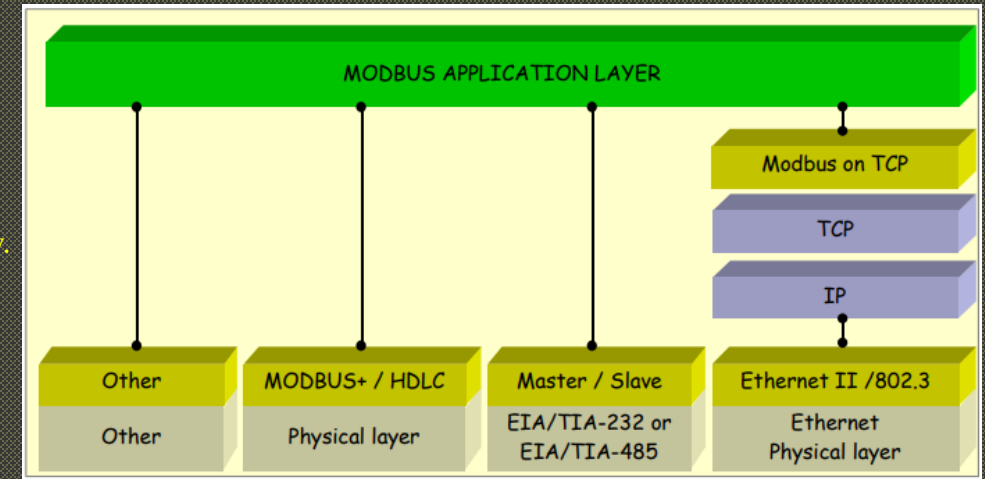
- Modbus operates at the Application Layer (Layer 7) of the Open Systems Interconnection (OSI) model.
- This means Modbus defines how data is formatted, addressed, and interpreted, but it does not define the physical medium, how data is transmitted electrically, or via Ethernet.
- Those lower-level details are handled by other layers.

Modbus RTU and ASCII use serial layers (RS-232/RS-485) for data transmission.
 Modbus TCP/IP uses Ethernet (IEEE 802.3) as its physical layer and TCP/IP for reliable packet delivery.

HDLC - High Level Data Link Control

Modbus Plus is a proprietary, high-speed network developed by Schneider Electric.

It is a peer-to-peer network using a token-passing mechanism and requires specialized hardware like a coprocessor for communication.



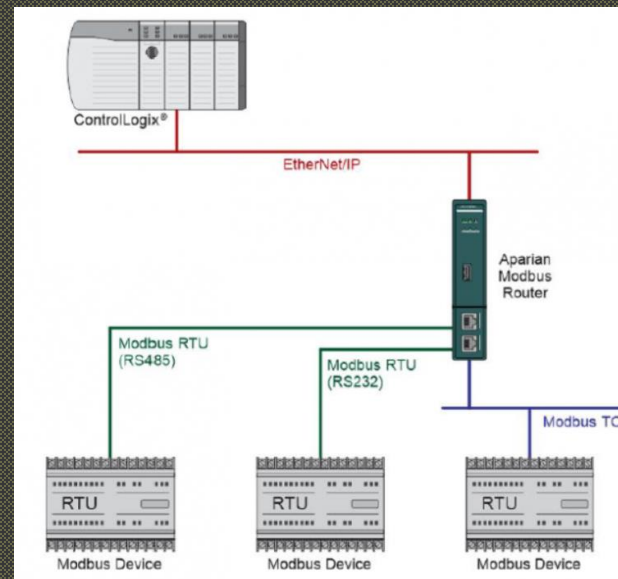
OSI Layer	Modbus Equivalent	Description
Layer 7 – Application	Modbus protocol	Defines messages, function codes, addressing
Layer 4 – Transport	TCP (for Modbus TCP/IP)	Ensures reliable delivery of messages
Layer 3 – Network	IP	Provides logical addressing (IP address)
Layer 2 – Data Link	Ethernet / Serial	Handles physical addressing and framing
Layer 1 – Physical	RS-485 / Ethernet cables	Transmits signals

7A.1 Industrial Communication and Modbus

- Modbus is applied across a wide range of industries and systems:

Industry	Application Example
Manufacturing	Machine control and process monitoring
Power & Energy	Energy meters, power quality analyzers
Building Automation	HVAC control, lighting, and security systems
Water/Wastewater	Pump control and level monitoring
Renewable Energy	Solar inverters, wind turbine monitoring
Transportation	Tunnel ventilation, rail signaling systems

In modern industrial networks, Modbus often serves as a bridge protocol - connecting legacy field devices (RTU) with modern Ethernet systems (TCP/IP) via gateways.



7A.2 Modbus Fundamentals

Modbus communication follows a client-server model, also referred to in older documentation as master-slave architecture.

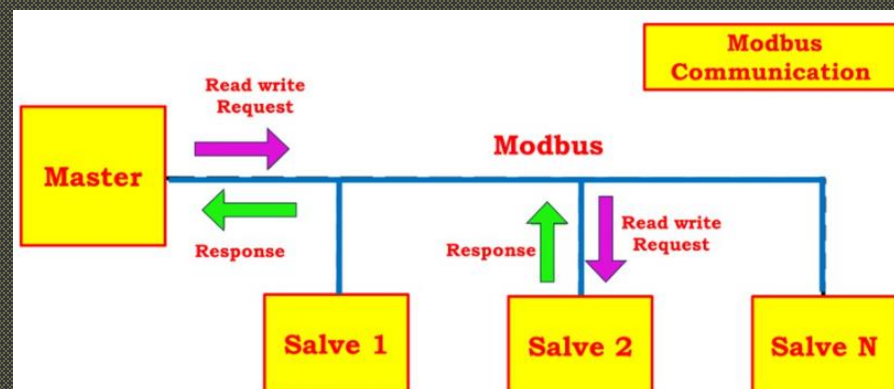
- Client (Master): Initiates all data transactions by sending a request message.
- Server (Slave): Responds to the request with the requested data or performs the commanded action.

In Modbus networks:

- Only the client can start communication.
- Servers cannot initiate communication; they only respond.
- Each server device must have a unique address (1-247 in RTU/ASCII; Unit ID in TCP/IP).



<https://www.prosoft-technology.com/Landing-Pages/Protocol/Modbus-and-Modbus-TCP-Protocol>



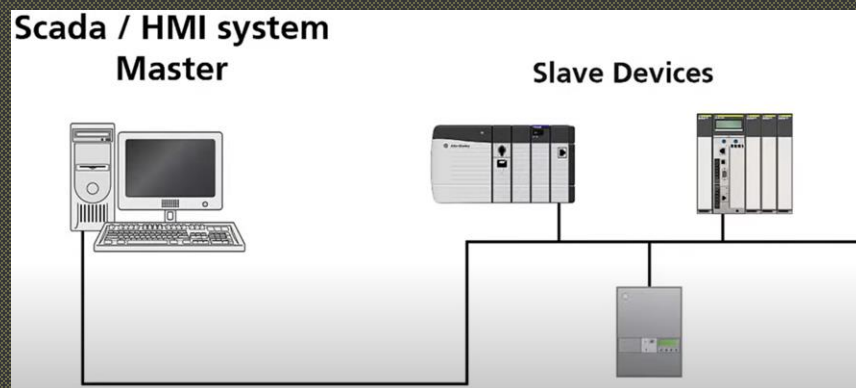
<https://automationforum.co/modbus-communication-protocol/>

7A.2 Modbus Fundamentals

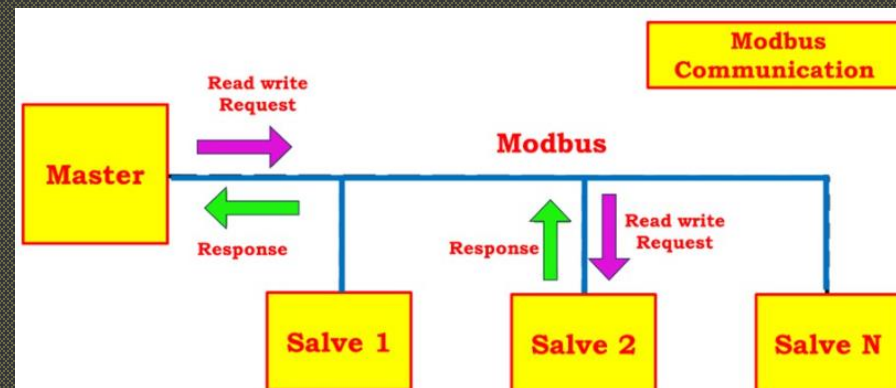
Each Modbus transaction consists of a request from the client and a response from the server. This mechanism ensures deterministic communication - every request corresponds to one response.

- The client sends a request: “Read Holding Register 40001.”
- The server interprets the message and retrieves the requested data.
- The server replies: “Value of Register 40001 = 1500.”
- The client processes or displays this data.

This predictable exchange is critical in control applications where timing and confirmation of data are essential.



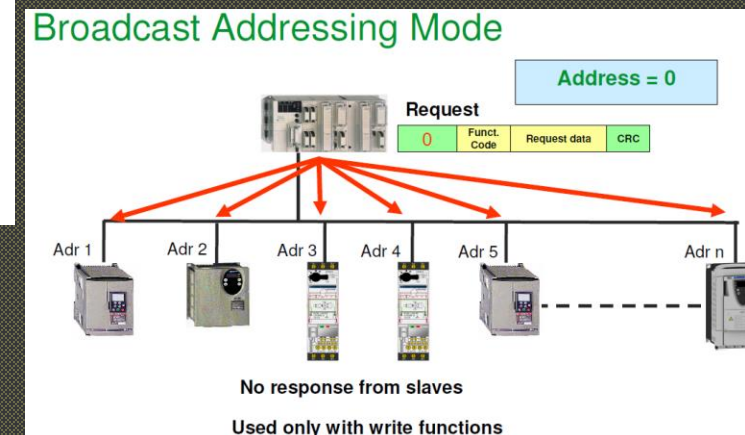
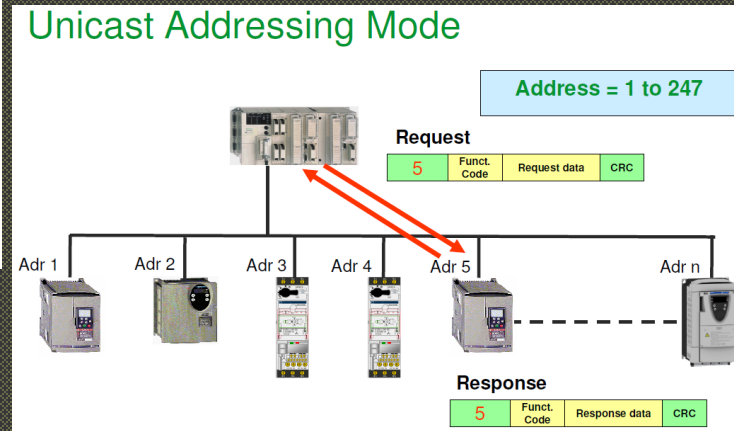
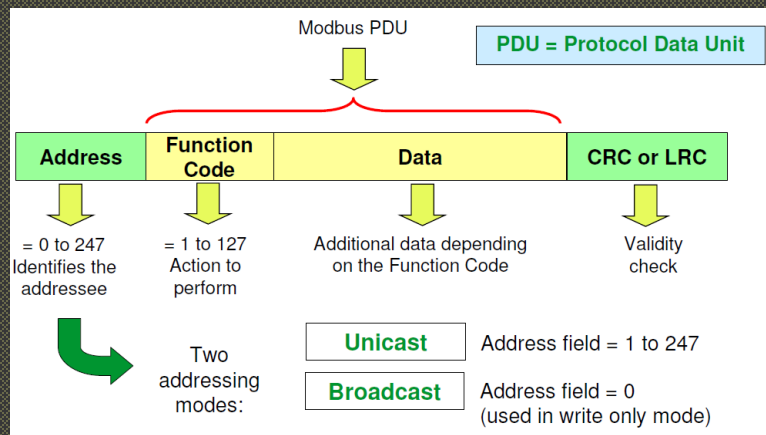
<https://www.prosoft-technology.com/Landing-Pages/Protocol/Modbus-and-Modbus-TCP-Protocol>



<https://automationforum.co/modbus-communication-protocol/>

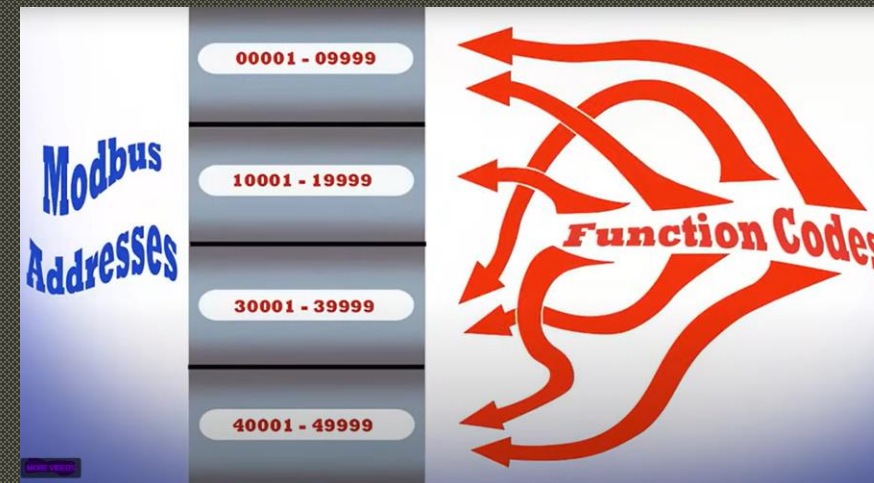
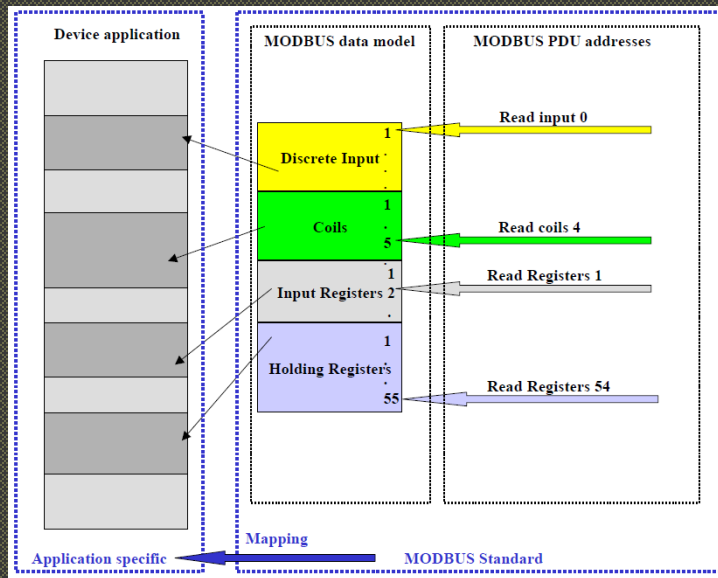
7A.2 Modbus Fundamentals

- Request
 - A master addresses a slave by placing the slave address in the address field of the message.
- Response
 - When the slave sends its response, it places its own address in this address field of the response to let the master know which slave is responding.



7A.2 Modbus Fundamentals

In Modbus, data is organized into four types of registers or “data models.” Each data type has its own address range and access rights (read-only or read/write).



<https://www.prosoft-technology.com/Landing-Pages/Protocol/Modbus-and-Modbus-TCP-Protocol>

When programming or configuring devices, these addresses are often shown without the leading digit (e.g., address 40001 becomes register 0 internally).

- Coils – single bit read/write outputs
- Discrete input – single bit read-only inputs
- Input register – 16 bit read-only analog sensor readings
- Holding register - memory block used for a variety of data types

Data Type	Access	Address Range	Typical Function Code	Description
Coils	Read/Write	00001–09999	01, 05, 15	Discrete output (e.g., motor ON/OFF)
Discrete Inputs	Read Only	10001–19999	02	Digital input (e.g., switch state)
Input Registers	Read Only	30001–39999	04	Analog input (e.g., temperature, voltage)
Holding Registers	Read/Write	40001–49999	03, 06, 16	Analog output or internal variables

The first digit (0, 1, 3, or 4) tells the Modbus software which function code to use, not the actual register number.

7A.2 Modbus Fundamentals

Modbus uses function codes to specify what action the client wants to perform. The following are commonly used:

Function Code	Action	Description
01	Read Coils	Read status of digital outputs
02	Read Discrete Inputs	Read status of digital inputs
03	Read Holding Registers	Read internal variables or analog outputs
04	Read Input Registers	Read analog input values
05	Write Single Coil	Set one digital output
06	Write Single Register	Set one holding register
15 (0F hex)	Write Multiple Coils	Set multiple digital outputs
16 (10 hex)	Write Multiple Registers	Set multiple holding registers

- Each response includes the same function code unless an error occurs, in which case the most significant bit of the function code is set to 1 (indicating an exception)
- When a Modbus server detects an error (e.g., invalid function, illegal address, or data value), it replies with an exception response.
- For example, if the client requests an invalid register, the server sends back a function code with the highest bit set and an exception code indicating the error type.

Exception Response

- Request: Function 03 (Read Holding Registers)
- Response: Function 83 (Error) + Exception Code 02 → Illegal Data Address

Code	Meaning
01	Illegal Function
02	Illegal Data Address
03	Illegal Data Value
04	Slave Device Failure

7A.2 Modbus Fundamentals

Linking Modbus addressing to real PLC I/O wiring

- A discrete input sensor is wired into a PLC discrete input module: connected to input channel 1 of slot 1 of the PLC rack.
- The PLC manufacturer uses the addressing format I:slot/bit (Allen-Bradley) or %I0.x (Siemens)

- Physical Wiring and PLC Addressing

Description	PLC Address	Meaning
Sensor wired to input 1	I:1/0	Rack 0, Slot 1, Bit 0

- When Modbus is enabled on this PLC (as a server/slave), the PLC firmware exposes its internal I/O data to Modbus using standard Modbus data models.
- Discrete inputs are mapped into the Modbus discrete input area (1xxxx range), which is read-only.

- Internal Mapping to Modbus Address Space

Data Type	Logical Modbus Address	Function Code	Description
Discrete Input	10001	02	First discrete input bit

- Modbus Address Representation

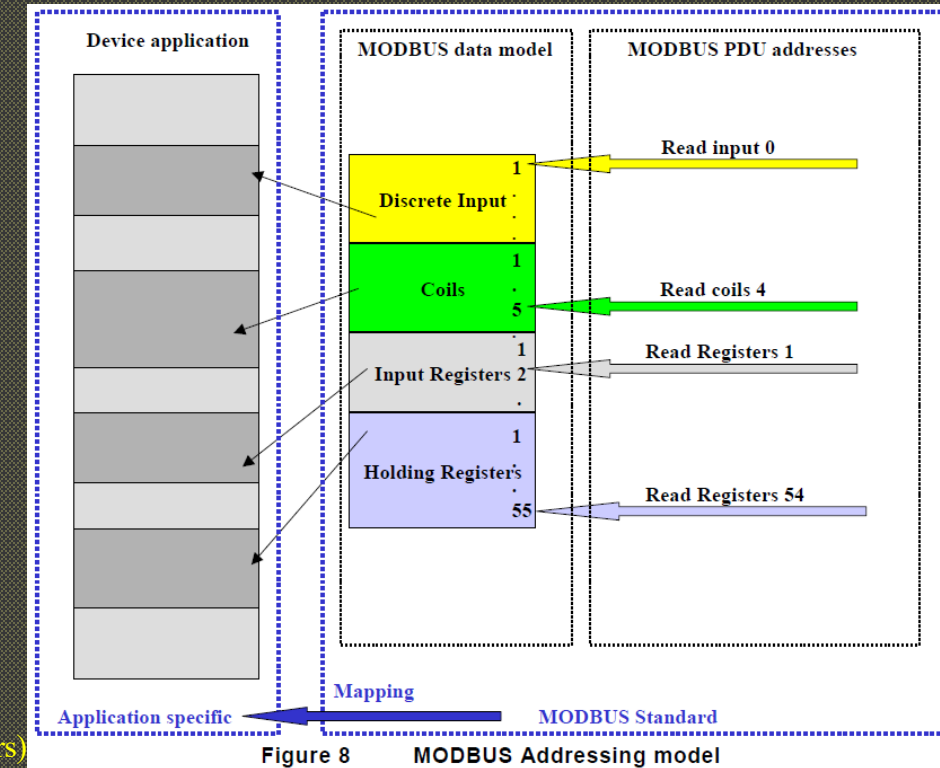
- Documentation Address (Human-readable):
 - Input = 10001 → This is what you will see in manuals, register maps, and HMI configuration tables.
- Internal Register Number (Zero-based):
 - Input = 0000 (hex) → This is what is actually used inside the Modbus frame - because the numbering starts from 0.

7A.2 Modbus Fundamentals

- **Device Application (Left Box)**
 - This represents the actual memory layout inside a physical device like a PLC, sensor, or actuator.
 - The memory here is application-specific. Each device manufacturer decides how data is stored internally.
 - The MODBUS protocol doesn't care how it's physically organized; it only needs to map this memory to a standardized model.
- **MODBUS Data Model (Middle Box)**
 - This is the standardized view of a device's data, and it is split into four main blocks: Discrete Input, Coils, Input Registers, and Holding Registers
 - Each block starts its numbering from 1, but internally these are zero-based (starts from 0 in the message)
- **MODBUS PDU Addresses (Right Box)**
 - This shows the actual address values used when communicating with the device over MODBUS.
- These are the zero-based addresses sent in the MODBUS message.
- Examples from the diagram:
 - Discrete Input 1 → Addressed as 0 in PDU → (Function code 02: Read Discrete Inputs)
 - Coil 5 → Addressed as 4 in PDU → (Function code 01: Read Coils or 05/15 to write)
 - Input Register 2 → Addressed as 1 in PDU → (Function code 04: Read Input Registers)
 - Holding Register 55 → Addressed as 54 in PDU → (Function code 03 or 06: Read/Write Holding Registers)

Data Type	Access	Address Range
Coils	Read/Write	00001–09999
Discrete Inputs	Read Only	10001–19999
Input Registers	Read Only	30001–39999
Holding Registers	Read/Write	40001–49999

Function Code	Action
01	Read Coils
02	Read Discrete Inputs
03	Read Holding Registers
04	Read Input Registers
05	Write Single Coil
06	Write Single Register
15 (0F hex)	Write Multiple Coils
16 (10 hex)	Write Multiple Registers



- **Mapping Arrow (Bottom Blue Arrow)**
 - This indicates the process of mapping real data in the device to the MODBUS data model.
 - It ensures that data can be consistently accessed using MODBUS, no matter how it is physically stored.

7A.2 Modbus Fundamentals

Modbus Message Structure

- Every Modbus message contains key components that identify the source, destination, action, and data.

Request: [01][03][00][64][00][02][CRC]

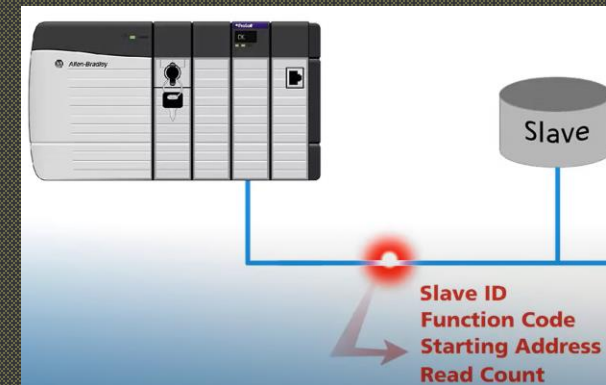
- 01 → Device address
- 03 → Function code (read holding registers)
- 00 64 → Starting address (100 decimal)
- 00 02 → Number of registers to read
- CRC → Error check value

If successful, the server replies with:

Response: [01][03][04][00][FA][01][2C][CRC]

- 04 → Number of bytes of data
- 00 FA → First register value (250)
- 01 2C → Second register value (300)

Field	Description
Device Address	Identifies the server/slave device
Function Code	Defines the requested action (e.g., read/write)
Data	Contains address, quantity, and values
Error Check (CRC)	Detects transmission errors



7A.2 Modbus Fundamentals

Modbus Message Structure

- Every Modbus message contains key components that identify the source, destination, action, and data.

RTU Packet Format:

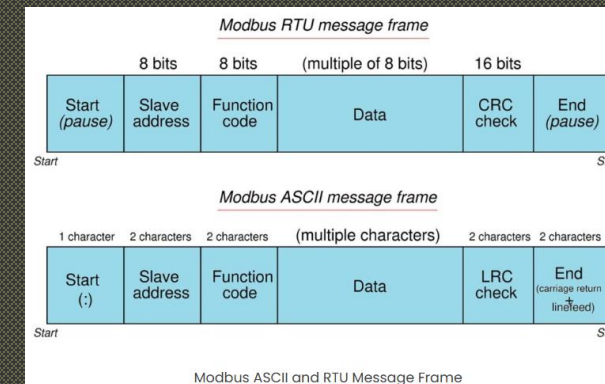
Address	Function Code	Data	CRC
1 byte	1 byte	Variable	2 bytes

ASCII Packet Format:

Start Code	Address	Function Code	Data	LRC	End of Line
1 char	2 chars	2 chars	Variable	2 chars	2 chars (CR LF)

CR – Carriage Return

LF – Line Feed



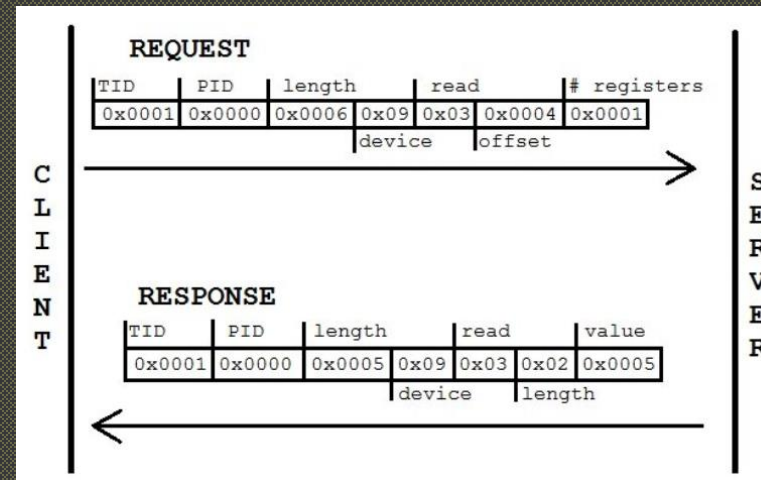
7A.2 Modbus Fundamentals

Modbus Message Structure

- Every Modbus message contains key components that identify the source, destination, action, and data.

TCP Packet Format:

Transaction Identifier	Protocol Identifier	Length	Unit Identifier	Function Code	Data
2 bytes	2 bytes	2 bytes	1 byte	1 byte	Variable



RESPONSE:

TID: 0x0001 (same as request, so the response matches this transaction)

PID: 0x0000 (same as request – always 0000 for Modbus)

Length: 0x0005 (5 bytes to follow)

Then we have: 0x09, 0x03, 0x02, 0x0005

Breaking down the PDU:

Unit Identifier: 0x09 – 1 byte

Function Code: 0x03 (same as request) – 1 byte

Byte Count: 0x02 (2 bytes of data follow, because we read 1 register which is 2 bytes) – 1 byte

Data: 0x0005 (the value of the register) – 2 bytes

REQUEST:

TID: 0x0001 (Transaction Identifier)

PID: 0x0000 (Protocol Identifier)

Length: 0x0006 (6 bytes to follow)

Then we have: 0x09, 0x03, 0x0004, 0x0001

Breaking down the PDU:

Unit Identifier (device): 0x09 – 1 byte

Function Code: 0x03 (Read Holding Registers) – 1 byte

Starting Address: 0x0004 (so register 40005? Remember, the protocol uses 0-based, but often documented as 40001+) – 2 bytes

Quantity: 0x0001 (read 1 register) – 2 bytes

7A.2 Modbus Fundamentals

Example of a Modbus Request

- Suppose an HMI or SCADA system wants to read the status of an input.
- It sends a Modbus Function Code 02 (Read Discrete Inputs) message.

- Example Modbus RTU Request (hexadecimal):

• [Slave Address] [Function Code] [Start Addr Hi] [Start Addr Lo] [Quantity Hi] [Quantity Lo] [CRC]

- So, the request looks like: 01 02 00 00 00 01 CRC_low CRC_high
- This tells the PLC:
- “Read 1 discrete input, starting from address 0000 (which corresponds to Modbus address 10001).”

- Example of Response from PLC

- If the input (sensor) is ON, the PLC returns:

• 01 02 01 01 CRC_low CRC_high

- If the input is OFF, the PLC returns:

• 01 02 01 00 CRC_low CRC_high

- The third byte (“01”) in the response means 1 byte of data, and the fourth byte contains the bit values of those inputs.

RTU Packet Format:

Address	Function Code	Data	CRC
1 byte	1 byte	Variable	2 bytes

Data (Variable Length): The parameters required by the function code. For a read, this is the starting address and quantity. For a write, it is the starting address and the data to be written.

Field	Value	Meaning
Slave Address	01	PLC node ID 1
Function Code	02	Read Discrete Inputs
Start Addr Hi	00	High byte of 0000
Start Addr Lo	00	Low byte of 00 00 (first discrete input)
Quantity Hi	00	High byte of 0001
Quantity Lo	01	Read 1 bit of 00 01
CRC	—	Calculated checksum

7A.2 Modbus Fundamentals

Modbus Timing and Communication Control

- In Modbus RTU protocol, frames are transmitted as continuous byte streams without explicit start/end markers.
- Instead, frame boundaries are defined by timing: a 3.5-character time gap serves as the minimum silent interval (quiet period on the line) between consecutive frames.
- This gap alerts all devices that the prior frame has ended and a new one may begin.
 - A "character time" (T_{char}) is the duration to transmit one full byte (character), including start bit (1), data bits (8), parity bit (optional, 1), and stop bit (1): totaling ~10 bits.
 - Formula: $T_{\text{char}} = 10 / \text{Baud Rate}$ (in seconds; e.g., at 9600 bps, $T_{\text{char}} \approx 1.04 \text{ ms}$).
- Examples: The gap duration scales inversely with baud rate:

RTU Packet Format:

Address	Function Code	Data	CRC
1 byte	1 byte	Variable	2 bytes

Baud Rate (bps)	Character Time (ms)	3.5-Character Gap (ms)
9600	1.04	3.64
19200	0.52	1.82
38400	0.26	0.91

If the line is idle for ≥ 3.5 character times after a frame, devices recognize it as complete, enabling reliable multi-device bus communication.

7A.2 Modbus Fundamentals

Modbus Timing and Communication Control

- Modbus RTU relies on precise timing for frame separation due to the absence of explicit start/end delimiters (unlike ASCII mode).
- The 3.5-character silence gap is essential to delineate consecutive messages - devices use it to detect frame boundaries.
- Consequences of Timing Violations
 - Too short a gap: Slaves may merge frames, leading to CRC errors or invalid responses.
 - Too long a gap: Masters may timeout, disrupting communication.
 - Modbus devices are designed to monitor and enforce this silence for reliable bus operation.
- Comparison with Modbus TCP/IP
 - TCP layer automates timing via ordered delivery, error retransmission, and integrity checks.
 - No silence gap required; frames are naturally segmented by TCP streams.
 - Advantage: TCP/IP is more robust and less timing-dependent, suiting Ethernet-based modern industrial networks.

7A.2 Modbus Fundamentals

Example of a Modbus Request

- An HMI (client) requests temperature data from a PLC (server) that measures it using an analog sensor connected to input register 30005.

Hex Value	Field Name	Description
01	Slave Address	Identifies the PLC as slave #1 (hex 01). In a multi-device bus, this ensures only the target responds; others ignore it.
04	Function Code	Read Input Registers (FC 04): Requests values from read-only input registers (3xxxx area).
00	Starting Address (High Byte)	Part of the 16-bit starting register address: 00 04 = 0x0004 (decimal 4). In Modbus addressing, this is the internal offset (30001 = 0x0000, so 30005 = 0x0004). The "3" in 30005 indicates the register type, not part of the transmitted value.
04	Starting Address (Low Byte)	Completes the address: Reads from register 30005 onward.
00	Quantity of Registers (High Byte)	Part of the 16-bit quantity: 00 01 = 0x0001 (1 register). Max per request: 125 (to limit bandwidth).
01	Quantity of Registers (Low Byte)	Specifies exactly 1 consecutive 16-bit register (2 bytes of data expected in response).
[CRC Low] [CRC High]	CRC-16 Checksum	Error Detection Polynomial Server verifies; mismatch triggers retry.

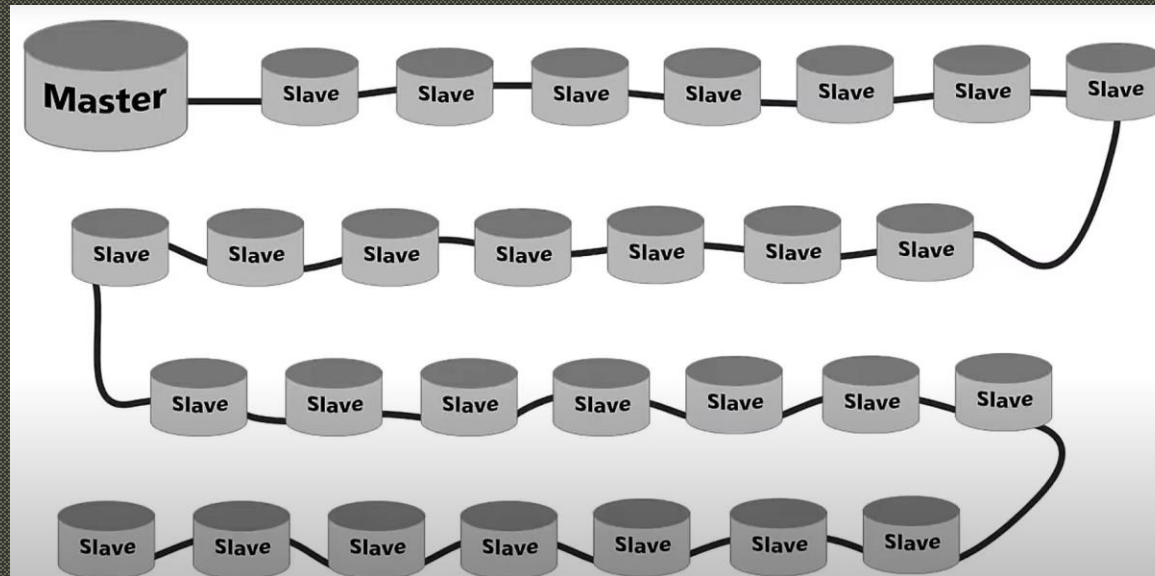
Hex Value	Field Name	Description
01	Slave Address	Echoes the request's slave ID for confirmation (though masters often ignore it).
04	Function Code	Echoes FC 04 to confirm success. (Exception: High bit set, e.g., 84 for error.)
02	Byte Count	Number of data bytes following: 2 bytes (1 register × 2 bytes/register). Always even; max 250 bytes.
0B	Register Value (High Byte)	Part of the 16-bit data: 0B B8 = 0x0BB8 (decimal 3000). Big-endian. This is the raw value from the sensor (e.g., scaled ADC output).
B8	Register Value (Low Byte)	Completes the value. In hex: 0B (11 decimal) × 256 + B8 (184) = 3000.
[CRC Low] [CRC High]	CRC-16 Checksum	Error Detection Polynomial Client verifies; mismatch triggers retry.

- Request (from HMI – Client/Master):
 - Function: 04 (Read Input Registers)
 - Address: 30005 (register 4 internally)
 - Quantity: 1
 RTU frame: [01][04][00][04][00][01][CRC]

- Response (from PLC – Server/Slave):
 - Value = 0x0BB8 (decimal 3000 → 30.00 °C if scaled by 0.01)
 - Frame: [01][04][02][0B][B8][CRC]

7A.3 Modbus Serial Communication (RTU and ASCII)

- In early industrial automation systems, serial communication was the standard method for connecting programmable logic controllers (PLCs), sensors, actuators, and other control devices.
- Before Ethernet became widespread, networks such as RS-232, RS-422, and RS-485 formed the foundation of industrial data exchange.
- The Modbus RTU and Modbus ASCII protocols were developed to operate over these serial links.
- Serial communication transmits data bit-by-bit over a single communication channel, using timing synchronization between devices. It is simple, cost-effective, and reliable for short to medium distances.



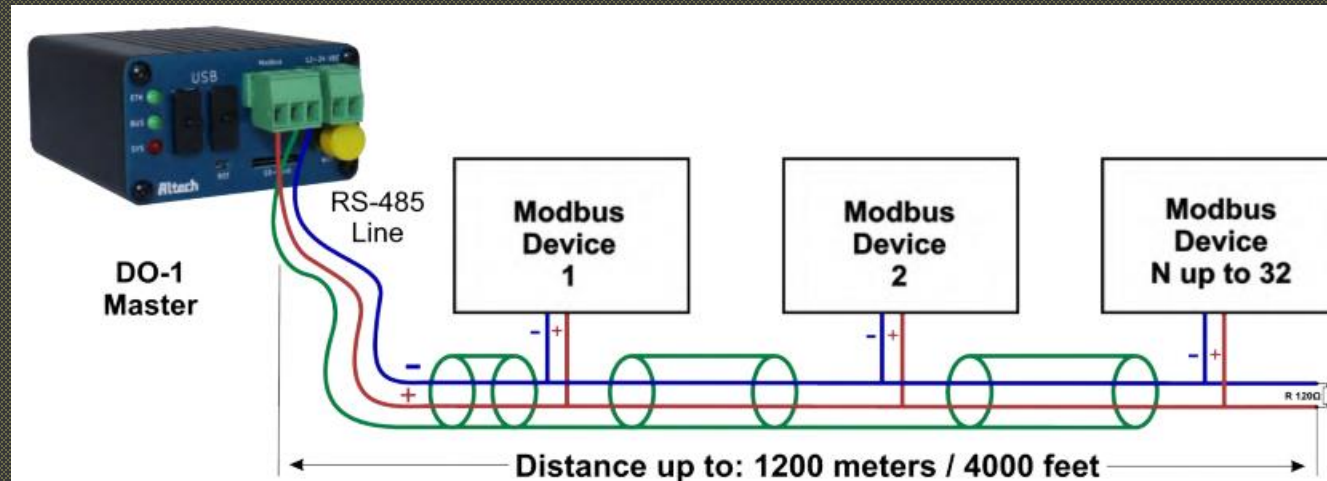
<https://www.prosoft-technology.com/Landing-Pages/Protocol/Modbus-and-Modbus-TCP-Protocol>

7A.3 Modbus Serial Communication (RTU and ASCII)

- In early industrial automation systems, serial communication was the standard method for connecting programmable logic controllers (PLCs), sensors, actuators, and other control devices.
- Before Ethernet became widespread, networks such as RS-232, RS-422, and RS-485 formed the foundation of industrial data exchange.
- The Modbus RTU and Modbus ASCII protocols were developed to operate over these serial links.
- Serial communication transmits data bit-by-bit over a single communication channel, using timing synchronization between devices. It is simple, cost-effective, and reliable for short to medium distances.

Modbus RTU:

- Every data item is encoded in one byte (8 bits)
- Achieves higher throughput & reliability
- Used with PLC / SCADA



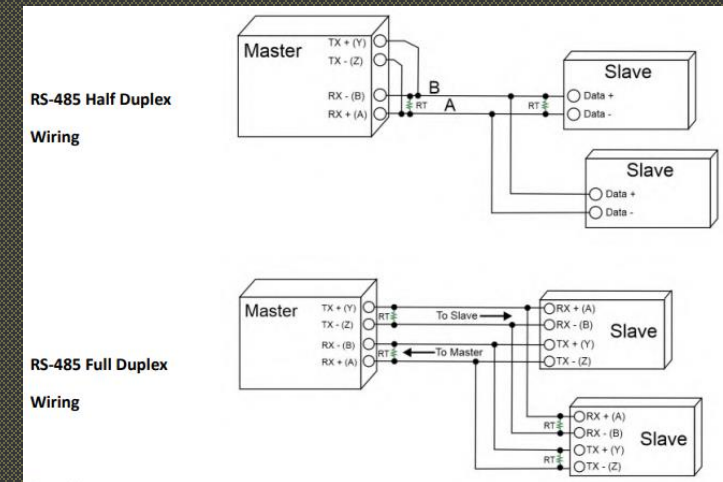
Modbus ASCII:

- Every data item is encoded in two ASCII characters (7 bits)
- Allows time intervals of up to 1 second to occur between characters without causing an error.
- Used with Modem / Barcodes.

7A.3 Modbus Serial Communication (RTU and ASCII)

- The Modbus protocol itself defines the data structure and communication rules, but not the physical wiring.
- The most common hardware layers used with Modbus are:

Standard	Description	Typical Use	Max Distance	No. of Devices
RS-232	Point-to-point connection between two devices	PC ↔ PLC	15 m	2
RS-422	Differential signaling for noise immunity	Longer single-master links	1 200 m	≤ 10
RS-485	Balanced differential bus supporting multiple devices	Industrial networks (Modbus RTU)	1 200 m	32 (or more with repeaters)



<https://altechmexico.com/img/cms/DO-1/DO-1-ENG/Modbus%20Reference%20Guide%20ENG.pdf>

RS-485 is the most popular physical layer for Modbus RTU networks.

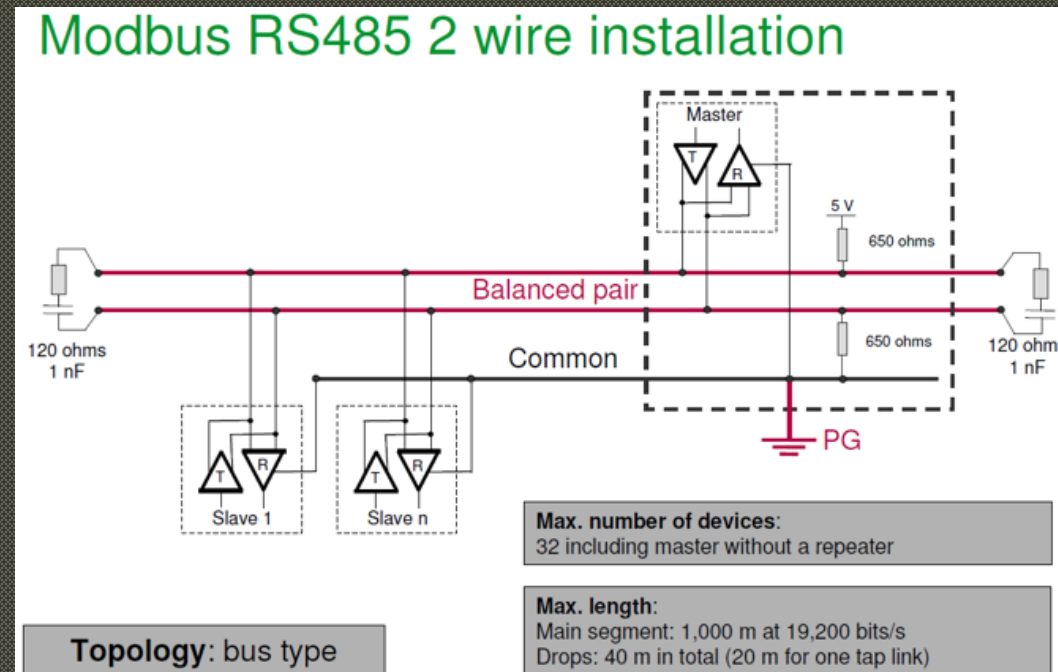
It supports half-duplex communication, meaning only one device transmits at a time, and uses differential voltage signaling for noise resistance in industrial environments.

7A.3 Modbus Serial Communication (RTU and ASCII)

- Each slave has a transceiver (T/R symbol) to send and receive data.
- Termination Resistors: 120 ohms at the Master and last Slave (or the end of the bus) match the characteristic impedance of the RS485 cable (typically 120 ohms) to prevent signal reflections, which can cause data errors.
- The two red lines labeled "Balanced pair" are the two wires of the RS485 network (typically labeled A and B or Data+ and Data-).
 - These wires carry the differential signals that allow robust communication over long distances and in noisy environments.
- The "Common" line connects all devices to a shared ground reference, ensuring consistent voltage levels.
- The network uses a linear bus topology, where all devices are connected along a single cable, with no branching beyond the specified drop lengths.

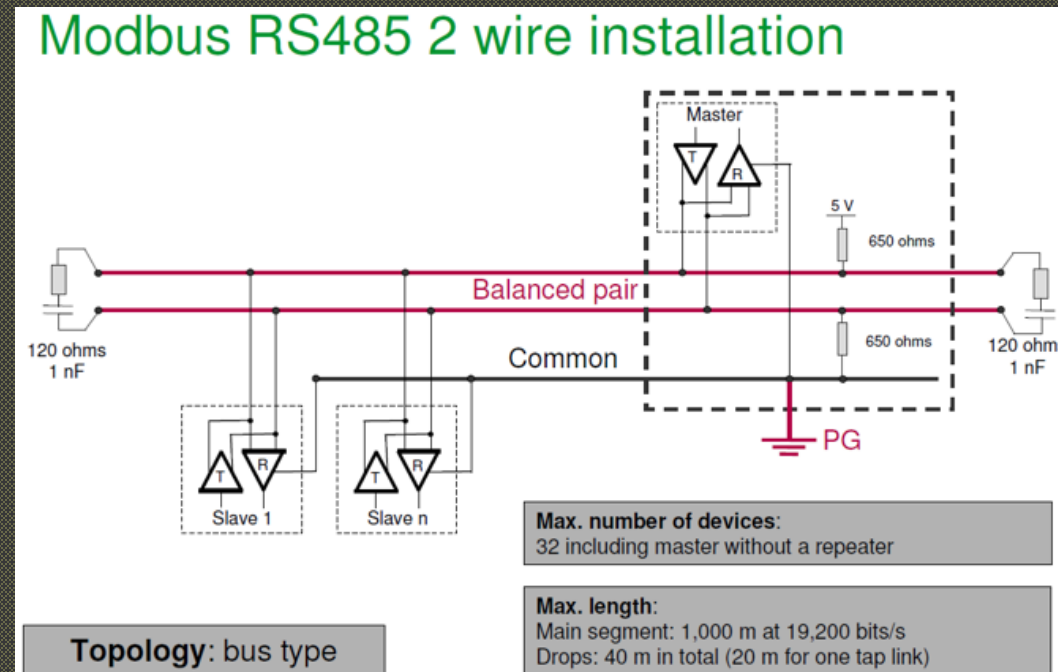
The Common line connects all device grounds together so they share the same reference voltage. This means:

- All devices “agree” on what 0 V means.
- The voltage levels on A and B stay consistent across the entire network.
- The RS-485 signals stay within the safe and readable range for all transceivers.



7A.3 Modbus Serial Communication (RTU and ASCII)

- Small 1 nF (nanofarad) capacitors are shown for filtering or stabilizing the signal by reducing high-frequency noise.
- Max. number of devices: 32, including the master, without a repeater.
- 650 ohms at the Master are pull-up or bias resistors associated with the RS485 transceiver's internal circuitry, ensuring a defined voltage level when the bus is idle.
- Max. length: 1,000 m at 19,200 bits/s. This is the maximum cable length for reliable communication at the specified baud rate (19,200 bps). Higher speeds reduce the maximum length (e.g., 300 m at 115,200 bps) to ensure signal integrity.



7A.3 Modbus Serial Communication (RTU and ASCII)

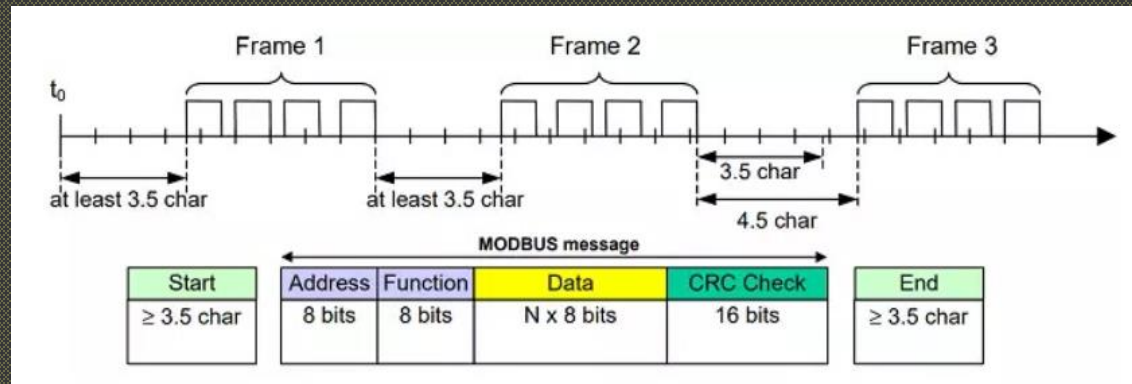
Modbus RTU Frame Format

- Modbus RTU (Remote Terminal Unit) is the binary version of the Modbus protocol. It is efficient and compact, using hexadecimal encoding for data.
- Each message is a sequence of bytes transmitted continuously without gaps longer than 3.5 character times between frames.
- Example: A master requests the value of a holding register (Function 03) from slave device 10 (0x0A).

Field	Size (bytes)	Description
Device Address	1	Identifies the slave device (1–247).
Function Code	1	Indicates the requested action (e.g., read, write).
Data	N	Contains parameters, register values, or addresses.
CRC (Cyclic Redundancy Check)	2	Error-checking field ensuring message integrity.

Field	Value (Hex)	Meaning
Device Address	0A	Slave #10
Function Code	03	Read Holding Registers
Data (Start Address)	006B	Register 107
Data (Quantity)	0003	Read 3 registers
CRC	7687	Error check

- Each Modbus server needs a unique address for identification.
- The address field is 1 byte, but only 1–247 are valid.
- 0 = broadcast;
- 248–255 = reserved.
- 247 is the maximum valid Modbus address by design, not by hardware limitation.



7A.3 Modbus Serial Communication (RTU and ASCII)

Modbus ASCII Frame Format

- Modbus ASCII (American Standard Code for Information Interchange) is a human-readable variant.
- Each byte is represented as two ASCII characters (e.g., the byte 0x4F is sent as “4” and “F”).
- It is slower but easier to debug using serial monitors.

Field	Description
Start	‘:’ (colon character)
Address	Two ASCII characters (0–9, A–F)
Function	Two ASCII characters
Data	Variable length
LRC (Longitudinal Redundancy Check)	Two ASCII characters
End	Carriage return + line feed (CR/LF)

Example Request: :010400000004F70D0A

Characters (Symbol)	Bytes (in Hex)	Meaning
:	3A	Start of Frame
01	30 31	Slave Address
04	30 34	Function Code
0000	30 30 30 30	Data Address
0004	30 30 30 34	Data Count (4 Registers)
F7	46 37	LRC Checksum
CR-LF	0D 0A	End of Frame

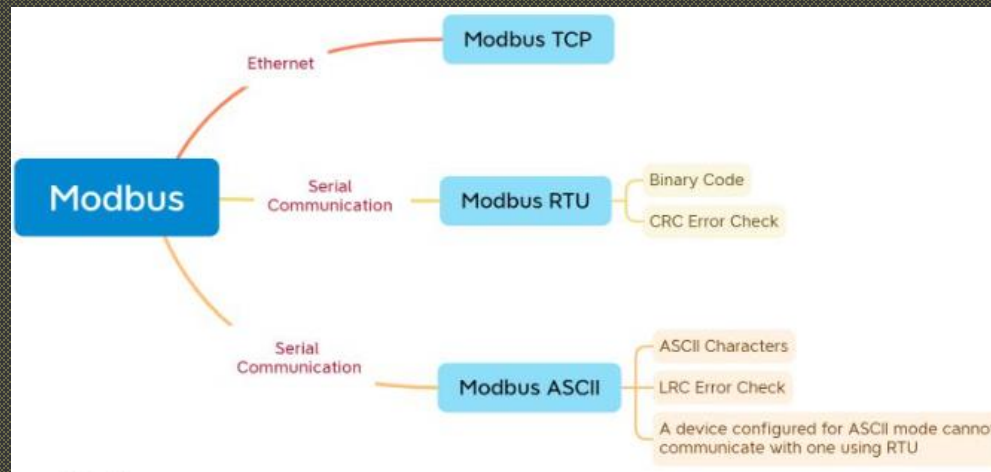
ASCII encoding	
code*	printed character
00	NUL †
0A	LF †
0D	CR †
30	0
31	1
32	2
33	3
34	4
35	5
36	6
37	7
38	8
39	9
3A	:
41	A
42	B
43	C
44	D
45	E
46	F

* hexadecimal format
† non-printing characters

7A.3 Modbus Serial Communication (RTU and ASCII)

CRC and LRC Error Checking

- Both Modbus RTU and ASCII include error-checking mechanisms to ensure message integrity.
 - CRC (Cyclic Redundancy Check) is used in Modbus RTU. It treats the entire message as a binary sequence and performs polynomial division to produce a 16-bit checksum. When the receiver recomputes CRC, both values must match for the message to be accepted.
 - LRC (Longitudinal Redundancy Check) is used in Modbus ASCII. It sums all byte values (excluding start/stop characters) and takes the two's complement to detect transmission errors.
 - These mechanisms protect against bit errors due to noise or interference, which are common in industrial environments.
- "Two's Complement" is a computer science trick to make the number negative.
- In practice, this step ensures that when the receiver adds everything together (the message bytes + the LRC byte), the total should be zero.
- For example, $4_{10} = 0100_2$; Converting/inverting 0s and 1s: 1011 ; Adding 1: $1011 + 1 = 1100$; Considering the most significant bit as a negative: $1100_2 = -4_{10}$



7A.3 Modbus Serial Communication (RTU and ASCII)

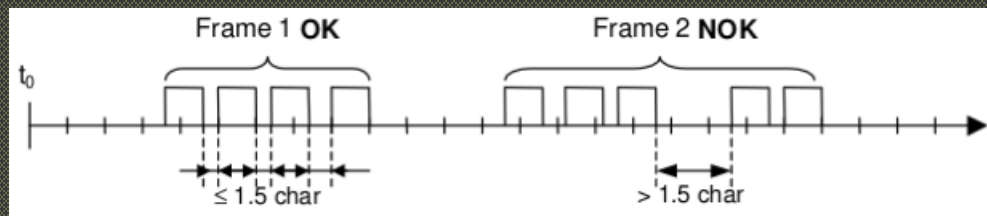
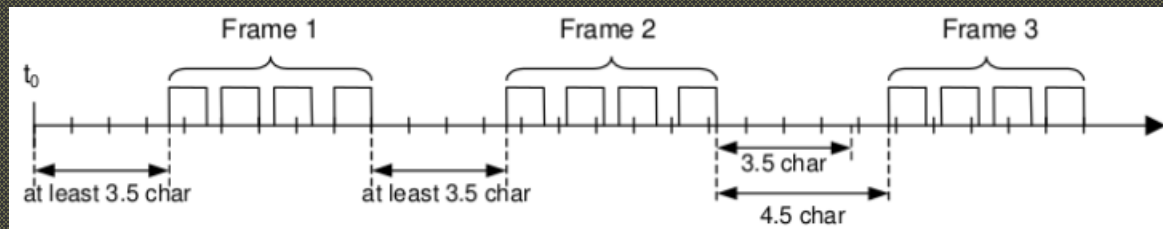
Communication Timing and Framing Rules

In Modbus RTU:

- The message frame must be continuous - if a gap > 3.5 -character times occurs, the frame is considered terminated.
- A gap of ≤ 1.5 - character times between bytes indicates an error.
- Timing is critical; baud rate determines character time (e.g., at 9600 bps, one character = 1.04 ms).

In Modbus ASCII:

- Timing is less strict because start (':') and end (CR/LF) markers clearly define message boundaries.



<https://electronics.stackexchange.com/questions/728659/a-question-about-silent-interval-in-modbus-rtu-framing>

START	ADDRESS	FUNCTION	DATA	LRC CHECK	END
1 CHAR :	2 CHARS	2 CHARS	n CHARS	2 CHARS	2 CHARS CRLF

<https://www.rfwireless-world.com/tutorials/modbus-protocol-tutorial-frame-formats>

Parameter	Modbus RTU	Modbus ASCII
Encoding	Binary (hexadecimal)	Text (ASCII)
Speed	Faster	Slower
Frame Start/End	Silence gaps	Start ':', End CR/LF
Error Check	CRC	LRC
Human Readability	Low	High
Typical Use	PLCs, sensors, controllers	Debugging, testing, legacy systems

7A.3 Modbus Serial Communication (RTU and ASCII)

Query Message from Master

The Master starts communication by sending a Query Message to a specific slave. This message includes:

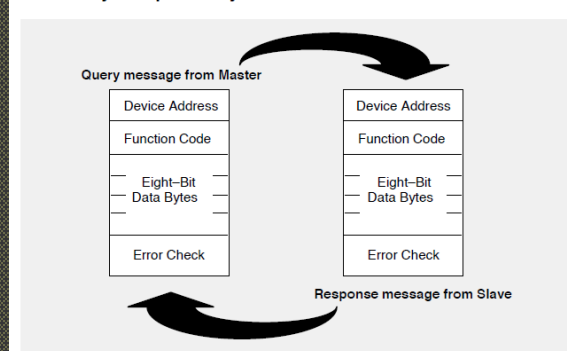
- Device Address
 - Tells which slave the master wants to talk to.
 - Example: If the address is 0x02, only device #2 will respond.
- Function Code
 - Tells the slave what kind of action to perform.
 - Example: 0x03 = Read Holding Registers, 0x01 = Read Coils.
- Data Bytes
 - Includes details like:
 - What to read/write
 - How many items
 - Starting address
 - Example: “Read 10 registers starting at address 100.”
- Error Check
 - Ensures data integrity (commonly a CRC or LRC checksum).
 - Used to detect if the message was corrupted during transmission.

Response Message from Slave

If the slave understands the request and can act on it, it sends a Response Message back. The structure is very similar to the request:

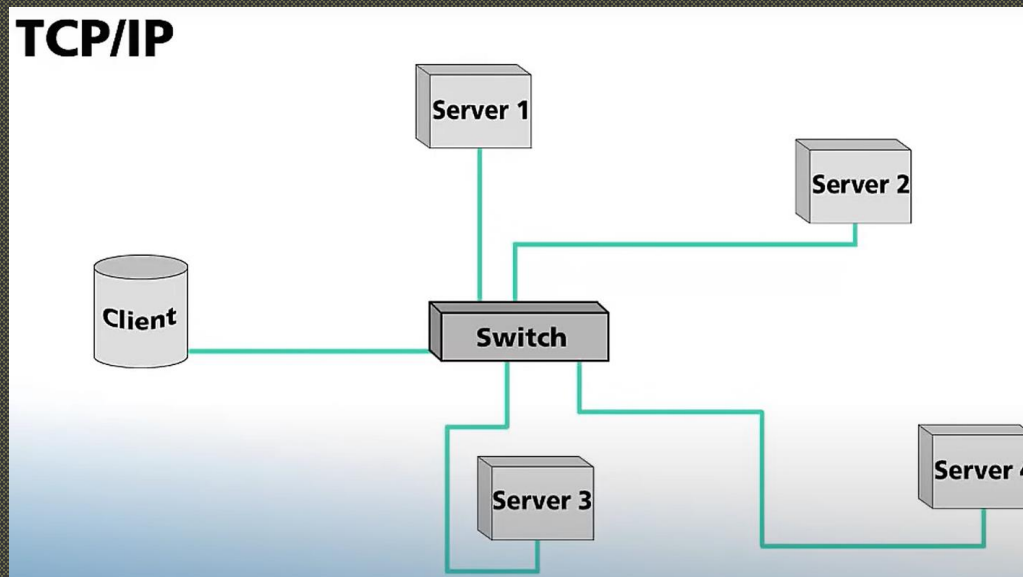
- Device Address
 - Echoed back to show which device is responding.
- Function Code
 - Echoed to indicate what the slave is responding to.
 - If an error occurred, the function code is modified (adds 0x80).
- Data Bytes
 - This is the actual requested data.
 - Example: If the master asked for 3 registers, this section will contain those values.
- Error Check
 - Again, this is used to verify that the message is received correctly.

The Query-Response Cycle

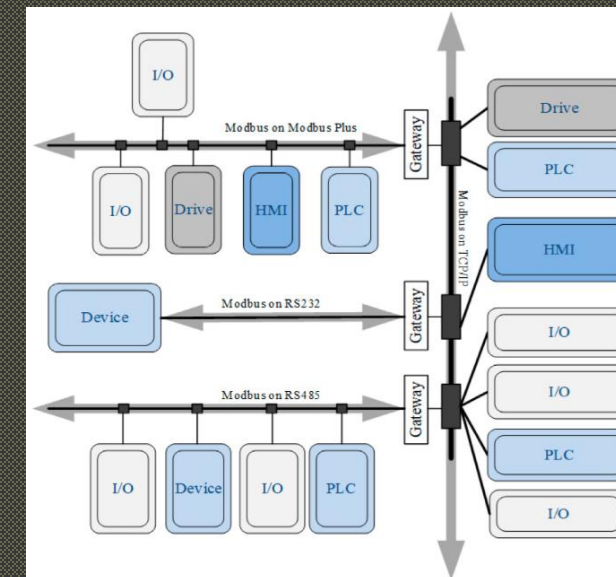


7A.4 Modbus TCP Communication

- With the widespread adoption of Ethernet technology, Modbus was extended to run over TCP/IP networks.
- Modbus TCP/IP uses the same application-layer protocol as Modbus RTU but encapsulates messages within TCP frames for reliable transmission.
- This allows Modbus devices (PLCs, HMIs, SCADA servers) to communicate using standard Ethernet infrastructure - switches, routers, and cables - integrating easily into enterprise networks.



<https://www.prosoft-technology.com/Landing-Pages/Protocol/Modbus-and-Modbus-TCP-Protocol>



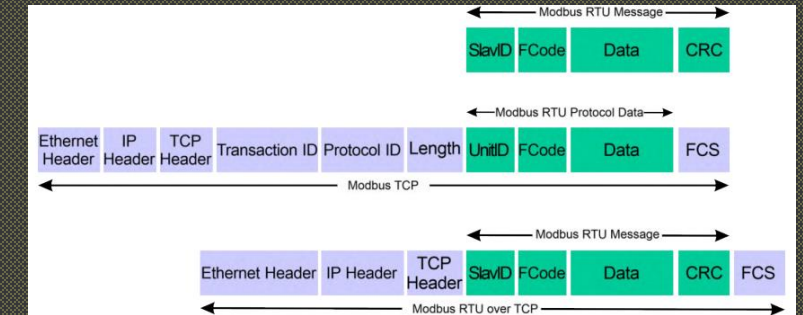
<https://www.mdpi.com/2227-9717/13/2/367>

7A.4 Modbus TCP Communication

- Ethernet is the physical and data-link layer standard used for local area networks (LANs).
- TCP/IP is the networking suite that provides addressing, routing, and reliable data delivery.

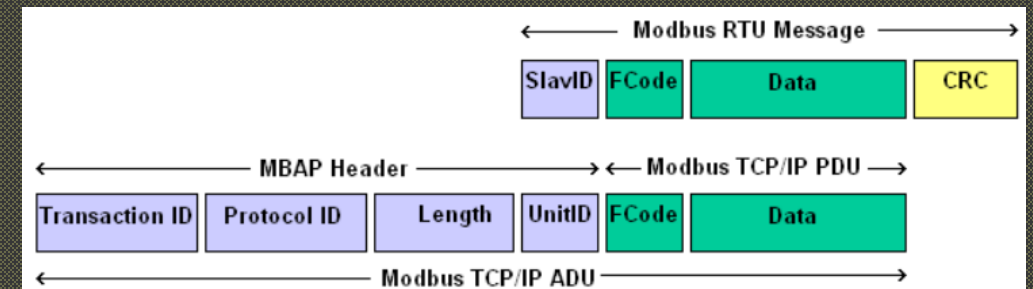
In Modbus TCP/IP:

- Modbus application data is placed inside a TCP frame.
- The TCP port 502 is reserved for Modbus communication.
- No CRC/LRC is needed - TCP already ensures data integrity via Frame Check Sequence.



<https://altechmexico.com/img/cms/DO-1/DO-1-ENG/Modbus%20Reference%20Guide%20ENG.pdf>

Field	Size (bytes)	Description
Transaction Identifier	2	Matches requests with responses.
Protocol Identifier	2	Always 0 for Modbus.
Length Field	2	Number of remaining bytes.
Unit Identifier	1	Slave address (used for bridging to serial networks).
Function Code	1	Action requested (e.g., 03 = Read Holding Registers).
Data	N	Parameters and register values.



<https://www.simplymodbus.ca/TCP.htm>

- MBAP Header Format (7 Bytes) – including 1 (Unit ID)
- After this header comes the PDU (Protocol Data Unit), which contains:
 - Function Code (1 byte) and Data (variable length)
 - 1 byte of (Unit ID) counts in the MBAP Length count

7A.4 Modbus TCP Communication

Example Modbus TCP/IP Message

- Request: A client requests to read 3 holding registers starting from address 107 (0x006B) from a PLC with Unit ID = 1
- 00 01 00 00 00 06 01 03 00 6B 00 03

Field	Value (Hex)	Meaning
Transaction ID	00 01	Request #1
Protocol ID	00 00	Modbus protocol
Length	00 06	6 bytes follow
Unit ID	01	Slave address
Function Code	03	Read Holding Registers
Data	00 6B 00 03	Start address 107, read 3 registers

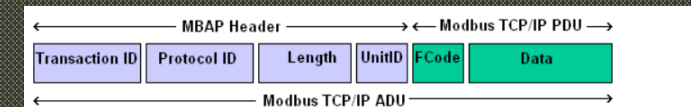
PDU (Protocol Data Unit) for Read Holding Registers (function 0x03):

- Function code: 0x03 → 1 byte
- Starting address: 0x006B → 2 bytes
- Quantity of registers: 0x0003 → 2 bytes

PDU length = 1 + 2 + 2 = 5 bytes

MBAP Length = 1 (Unit ID) + 5 (PDU) = 6 → 0x0006

Total bytes on wire = 7 (MBAP) + 5 (PDU) = 12 bytes



<https://www.simplymodbus.ca/TCP.htm>

Response: 00 01 00 00 00 09 01 03 06 02 2B 00 00 00 64

Field	Value (Hex)	Meaning
Transaction ID	00 01	Same as request
Protocol ID	00 00	Modbus protocol
Length	00 09	9 bytes follow
Unit ID	01	Slave address
Function Code	03	Read Holding Registers
Byte Count	06	3 registers × 2 bytes
Data	02 2B 00 00 00 64	Register values (e.g., 555, 0, 100)

When the server replies it sends:

- Function code: 0x03 → 1 byte
- Byte count: Number of registers × 2 → 3 × 2 = 6 → 1 byte (0x06)
- Data: 3 registers × 2 bytes each = 6 bytes (example values below)

PDU length = 1 (func) + 1 (byte count) + 6 (data) = 8 bytes

MBAP Length = 1 (Unit ID) + 8 (PDU) = 9 → 0x0009

37

7A.4 Modbus TCP Communication

- Modbus TCP/IP replaces the traditional master/slave terms with client/server:
 - The client (e.g., SCADA, HMI, or engineering workstation) initiates requests.
 - The server (e.g., PLC, energy meter, or I/O module) responds with data or confirmation.

Two common strategies:

1. Sequential Polling:

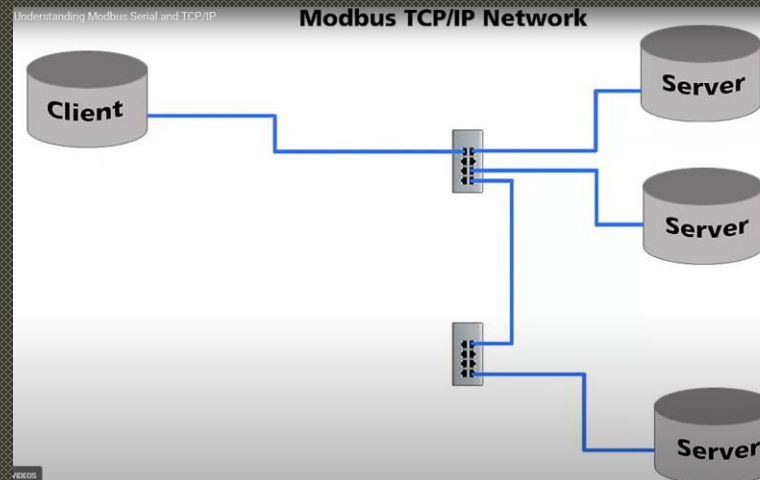
- The client polls devices one after another in sequence.
- Simple to implement but can increase latency as network grows.

2. Grouped Polling:

- Devices are grouped based on priority or data type.
- High-priority data (alarms, critical sensors) are polled frequently, while slow-changing data (temperature, flow) are polled less often.



<https://altechmexico.com/img/cms/DO-1/DO-1-ENG/Modbus%20Reference%20Guide%20ENG.pdf>



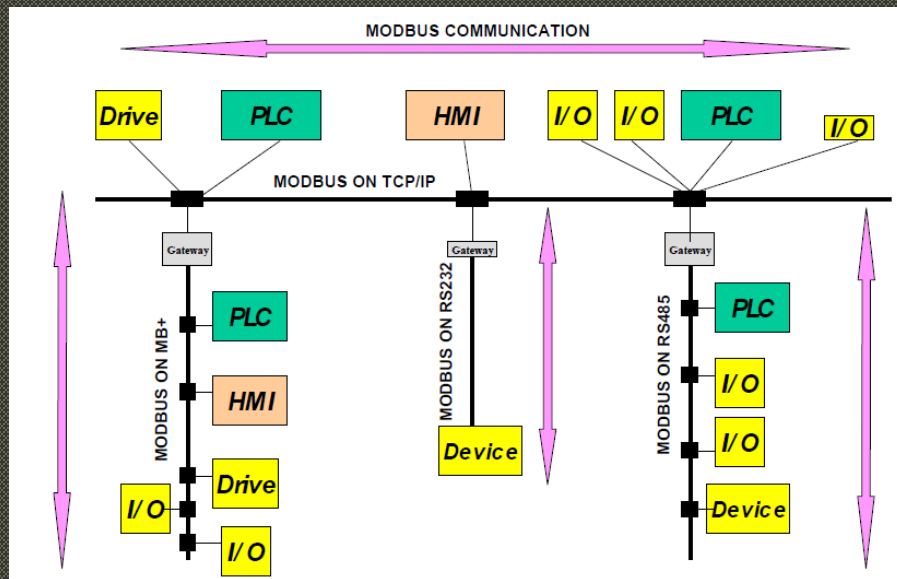
7A.4 Modbus TCP Communication

Many installations include both serial and Ethernet devices.

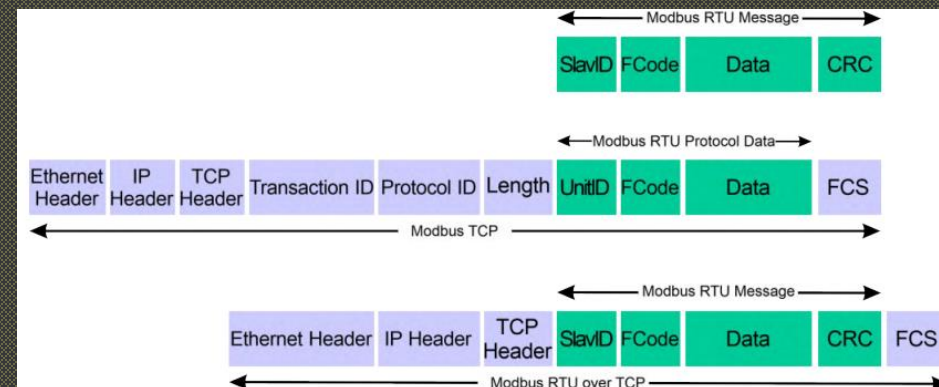
A Modbus gateway or protocol converter bridges between the two networks:

- Converts Modbus TCP requests from Ethernet clients into Modbus RTU messages for serial slaves.
- Translates responses back to TCP/IP format.

This interoperability allows legacy devices to remain in service while integrating with modern control systems.



Feature	Modbus RTU	Modbus TCP/IP
Physical Medium	RS-485 Serial	Ethernet
Error Detection	CRC	TCP Checksum
Addressing	1-247	IP Address + Unit ID
Max Devices	32 (per segment)	Practically unlimited
Typical Speed	≤115 kbps	≥100 Mbps
Typical Use	Local PLC networks	Plant-wide or remote systems



7A.5 Modbus Function Codes and Data Models

- The Modbus protocol defines a set of standardized function codes that determine what operation the client (or master) requests the server (or slave) to perform.
- Each function code represents a specific action, such as reading or writing data to registers or coils.
- Modbus uses a consistent data model to represent physical quantities like temperature, pressure, or motor status.
- This model abstracts device data into four types of memory areas or tables

Data Type	Table Name	Access	Typical Use	Address Range
Coils	Discrete Outputs	Read/Write	Control relays, motor starts	00001–09999
Discrete Inputs	Input Status	Read-only	Sensors, limit switches	10001–19999
Input Registers	Analog Inputs	Read-only	Sensors, analog feedback	30001–39999
Holding Registers	Analog Outputs	Read/Write	Setpoints, analog commands	40001–49999

Addressing Conventions

Although Modbus uses address ranges such as 40001 for holding registers, the protocol message does not include the leading digit.

For example:

- Register 40001 (first holding register) → Address 0000 in the message frame.
- Register 40010 → Address 0009.

Example: A request to read register 40010 will include the starting address = 0009 in the Modbus frame.

7A.5 Modbus Function Codes and Data Models

Common Function Codes

- The table below summarizes the most commonly used Modbus function codes:

Function Code (Hex)	Decimal	Description	Applicable Table
01	1	Read Coils	Coils
02	2	Read Discrete Inputs	Discrete Inputs
03	3	Read Holding Registers	Holding Registers
04	4	Read Input Registers	Input Registers
05	5	Write Single Coil	Coils
06	6	Write Single Register	Holding Registers
0F	15	Write Multiple Coils	Coils
10	16	Write Multiple Registers	Holding Registers

Example 1: Write Single Register - Set the output frequency of a variable frequency drive (VFD) to 60.00 Hz (value = 6000).

Example 2: Used when the client needs to send several values in one transaction, for example, writing multiple setpoints.

Field	Example (Hex)	Meaning
Slave Address	01	Device #1
Function Code	06	Write Single Register
Register Address	00 01	Holding Register 40002
Register Value	17 70	6000 decimal
CRC	D9 9B	Error check

Field	Example (Hex)	Meaning
Slave Address	01	Device #1
Function Code	10	Write Multiple Registers
Start Address	00 10	Starting register 4017
Quantity	00 02	Two registers
Byte Count	04	4 data bytes follow
Data	00 0A 00 14	10 and 20
CRC	C6 F0	Error check

7A.5 Modbus Function Codes and Data Models

Modbus TCP/IP Layer Integrity

- In TCP/IP mode, Modbus relies on the underlying TCP layer for error detection and retransmission.
- The TCP header includes checksums and sequence numbers to ensure that all data arrives intact and in the correct order.
- Therefore, no additional CRC or LRC field exists in Modbus TCP messages.

When a Modbus server (slave) detects a problem with a request, it sends an exception response.

This response is similar to a normal reply but with the function code modified - the most significant bit (MSB) is set to 1 (i.e., function code + 0x80).

For example:

- Request Function Code = 03
- Exception Response Function Code = 83

The next byte indicates the exception code, which explains the reason for the error.

Example of Exception Response

Request: A client sends a request to read from address 50000 using Function Code 03.

The device only supports addresses up to 49999.

Response: 01 83 02

Field	Value (Hex)	Meaning
Slave Address	01	Device address
Function Code	83	Exception response for function 03
Exception Code	02	Illegal Data Address

When no response is received within a certain time (typically 1-2 seconds), the client assumes a timeout has occurred.

The system then retries the request or flags a communication fault.

Timeout handling is essential in SCADA systems to prevent data gaps and to trigger alarms when communication is lost.

Exception Code	Name	Description
01	Illegal Function	Function code not supported by device.
02	Illegal Data Address	Requested address does not exist.
03	Illegal Data Value	Invalid data value or parameter.
04	Slave Device Failure	Internal device error.
05	Acknowledge	Request accepted, processing delayed.
06	Slave Device Busy	Device is performing another task.
08	Memory Parity Error	Memory check failed.
0A	Gateway Path Unavailable	Communication path through gateway unavailable.
0B	Gateway Target Failed to Respond	No response from target device.

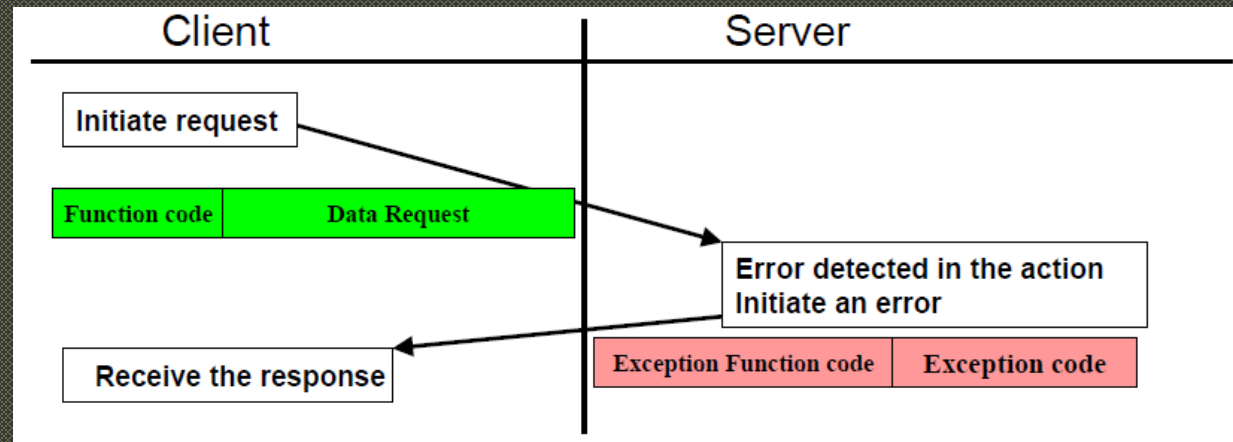
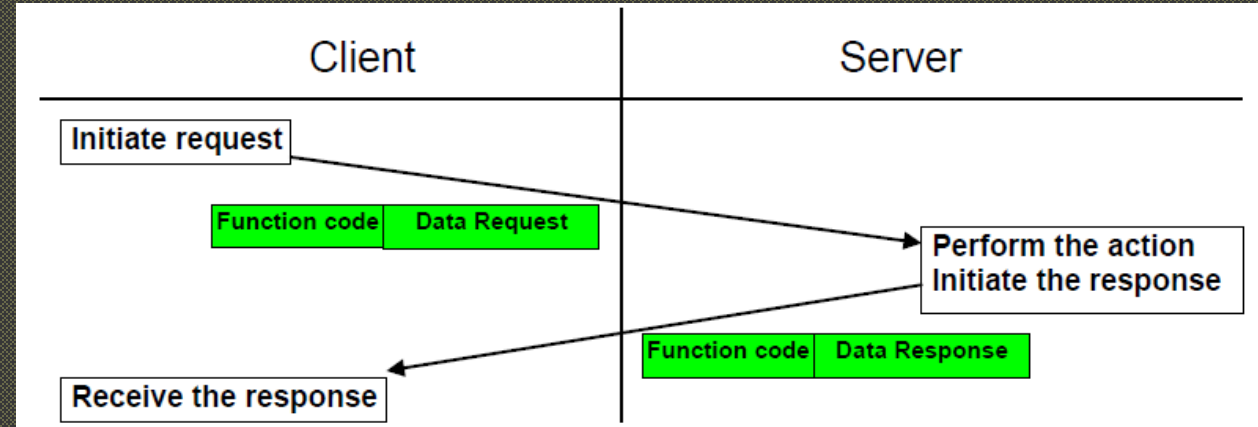
7A.5 Modbus Function Codes and Data Models

Client sends a request PDU to the server:

- Includes a function code (e.g., "Read Coils")
- Includes data like address, quantity of items, etc.

Server sends a response PDU back:

- Returns the requested data, or
- Returns an error code if something went wrong



7A.5 Modbus Function Codes and Data Models

Factory Motor Control

- Scenario: A factory has a Modbus-enabled PLC controlling a motor. The SCADA system wants to:
 - Write a new speed setpoint (e.g., 1500 RPM) and acceleration time (e.g., 10 seconds) to holding registers 40001 and 40002.
 - Read the motor's current speed, temperature, and status from holding registers 40003 to 40005.
 - Request: **Modbus TCP function code 23 (0x17 in hexadecimal) is known as Read/Write Multiple Registers.**
 - Write to registers 40001 (speed) and 40002 (acceleration time):
 - Register 40001: 1500 (0x05DC)
 - Register 40002: 1000 (0x03E8, representing 10.00 seconds)
 - Read registers 40003 (current speed), 40004 (temperature), and 40005 (status).
 - **Request PDU: 17 00 02 00 03 00 00 00 02 04 05 DC 03 E8**
 - 17: Function Code
 - 00 02: Read Starting Address (2, for 40003)
 - 00 03: Quantity to Read (3 registers)
 - 00 00: Write Starting Address (0, for 40001)
 - 00 02: Quantity to Write (2 registers)
 - 04: Byte Count ($2 \times 2 = 4$ bytes)
 - 05 DC 03 E8: Write Data (1500 for 40001, 1000 for 40002)
 - PLC Action:
 - The PLC writes 1500 to register 40001 (sets speed to 1500 RPM) and 1000 to register 40002 (sets acceleration time to 10 seconds).
 - It then reads registers 40003 to 40005:
 - 40003: Current speed = 1450 RPM (0x05AA)
 - 40004: Temperature = 3500 (0x0DAC, representing 35.00°C)
 - 40005: Status = 1 (0x0001, motor running)
 - Response:
 - **Response PDU: 17 06 05 AA 0D AC 00 01**
 - 17: Function Code
 - 06: Byte Count (3 registers $\times$ 2 bytes = 6 bytes)
 - 05 AA 0D AC 00 01: Read Data (1450 for 40003, 3500 for 40004, 1 for 40005)
- **SCADA Interpretation:**
 - **The SCADA system sees:**
 - Current speed: 1450 RPM (slightly below setpoint, maybe still adjusting).
 - Temperature: 35.00°C.
 - Status: Motor is running (1).
 - This confirms the motor is responding to the new setpoint and provides real-time feedback.
 - **Overlap Example:** If the client writes to 40001 and reads 40001 - 40002, the read will return the newly written value for 40001 (e.g., 1500) and the current value of 40002.
- Common scaling factors are:**
- x100 (two decimal places): 10.00 $\rightarrow$ 1000
 - x10 (one decimal place): 10.0 $\rightarrow$ 100
 - x1000 (three decimal places): 10.000 $\rightarrow$ 10000

7A.6 Modbus Security

This demonstrates how Wireshark decodes a Modbus TCP request and response, turning raw bytes into human-readable information. Click on a packet to see its details.

Time	Source	Destination	Protocol	Length	Info
0.000000	192.168.1.10	192.168.1.100	MODBUS/TCP	66	Query: Trans: 1; Unit: 1, Func: 3 (Read Holding Registers)
0.002134	192.168.1.100	192.168.1.10	MODBUS/TCP	69	Response: Trans: 1; Unit: 1, Func: 3 (Read Holding Registers)

Modbus TCP

00 01 00 00 00 06 01 ... MBAP Header

Transaction Identifier: 1

Protocol Identifier: 0

Length: 6

Unit Identifier: 1

Modbus

03 00 64 00 02 ... PDU

Function Code: Read Holding Registers (3)

Reference Number: 100 (Address 40101)

Word Count: 2

https://www.youtube.com/watch?v=3Sflf_yG5sY

7A.6 Modbus Security

Modbus Is Weak: Real-World Example

Imagine a factory with a Modbus TCP network controlling a conveyor belt. The system sends commands like “Set speed to 500 RPM” from a control room computer (client) to a PLC (server) over an Ethernet network.

- **Interception Risk:** A hacker on the same network could use a tool to “sniff” the traffic and see the command “500 RPM.” Since it’s not encrypted, they could read it or even send a fake command like “Set speed to 0 RPM” to stop the conveyor.
- **No Authentication:** The PLC accepts the command without checking if it’s from the real control room or a hacker, because Modbus doesn’t require a “password” or ID.
- **Tampering Risk:** The hacker could change the command mid-transit to “5000 RPM,” overloading the motor, and the PLC wouldn’t know the data was altered.

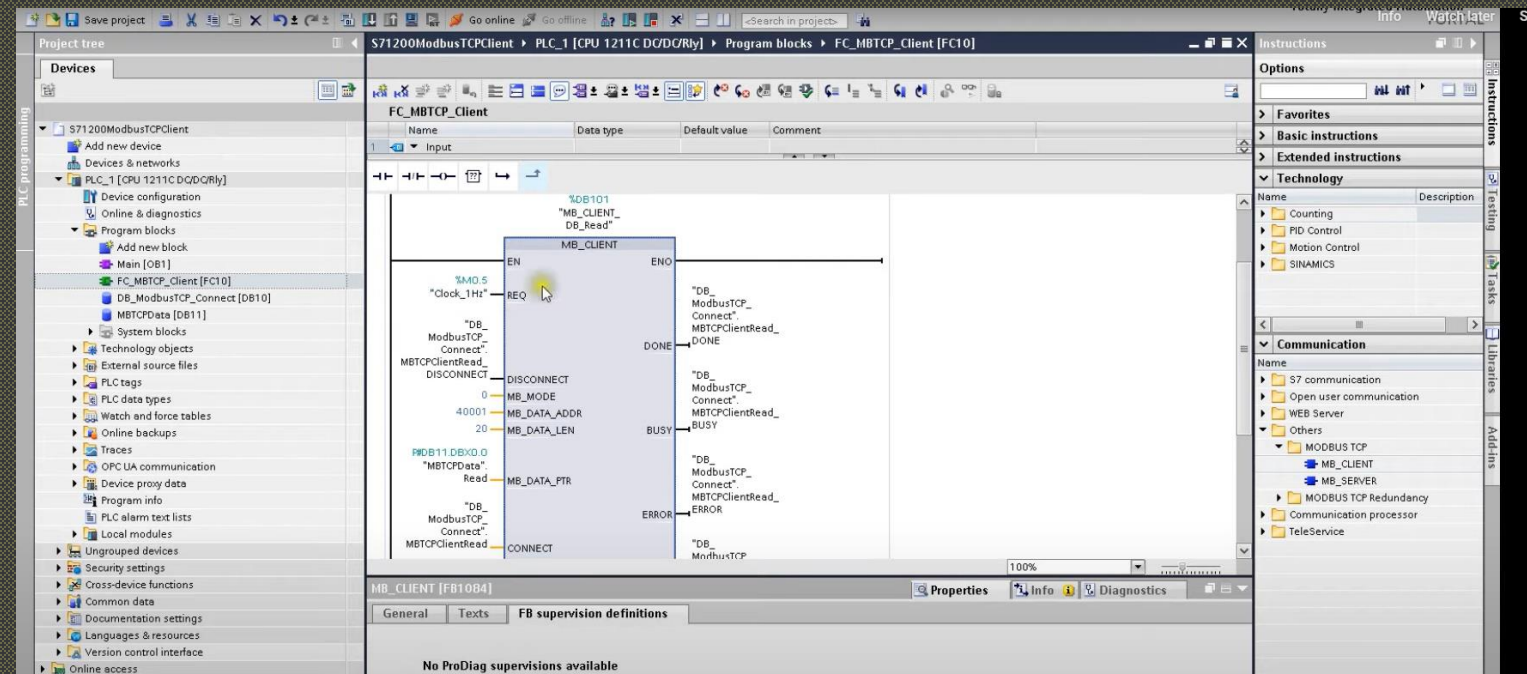
Solutions:

- **Use a VPN:**
 - A VPN encrypts all Modbus TCP data between the control room and the PLC, making it unreadable to outsiders.
 - **Example:** The factory sets up a VPN between the control room and the PLC. Now, when the command “Set speed to 500 RPM” is sent, it’s scrambled into code (e.g., “X7K9P2”) that only the PLC can decode with the VPN key. A hacker sees only gibberish, like a locked box they can’t open.
 - **Benefit:** Even if intercepted, the data is useless without the VPN key.
- **Use Firewalls:**
 - A firewall restricts access to the Modbus network, allowing only trusted devices to connect.
 - **Example:** The factory installs a firewall that only lets the control room computer (IP address 192.168.1.10) talk to the PLC. A hacker’s laptop (IP 192.168.1.100) is blocked, like a security guard stopping an uninvited guest at the factory gate.
 - **Benefit:** Prevents unauthorized devices from sending commands, even if they know the Modbus protocol.
- **Switch to Modbus Secure:**
 - Modbus Secure (e.g., Modbus TLS) adds encryption, authentication, and integrity checks to the protocol. It is an update to Modbus TCP designed to address security flaws.
 - **Example:** The factory upgrades to Modbus TLS. When sending “Set speed to 500 RPM,” the command is encrypted, and the PLC verifies the control room’s digital “signature” before acting. If a hacker sends a fake command, the PLC rejects it because the signature doesn’t match.
 - **Benefit:** Combines encryption (sealed letter) with authentication (verified sender), making it much harder to hack.

7A.7 Modbus in Siemens and Allen-Bradley PLCs

The Siemens PLCs support Modbus TCP communication through built-in function blocks in the TIA Portal software.

- The MB_CLIENT instruction enables the PLC to operate as a Modbus TCP client, initiating requests to read or write data from a remote Modbus TCP server.
- The MB_SERVER instruction allows the PLC to act as a Modbus TCP server, responding to requests from a Modbus client.
- Both blocks handle all Modbus message formatting and TCP/IP transport automatically.
- Communication occurs through the CPU's integrated PROFINET interface, which serves as the physical Ethernet connection for Modbus TCP.

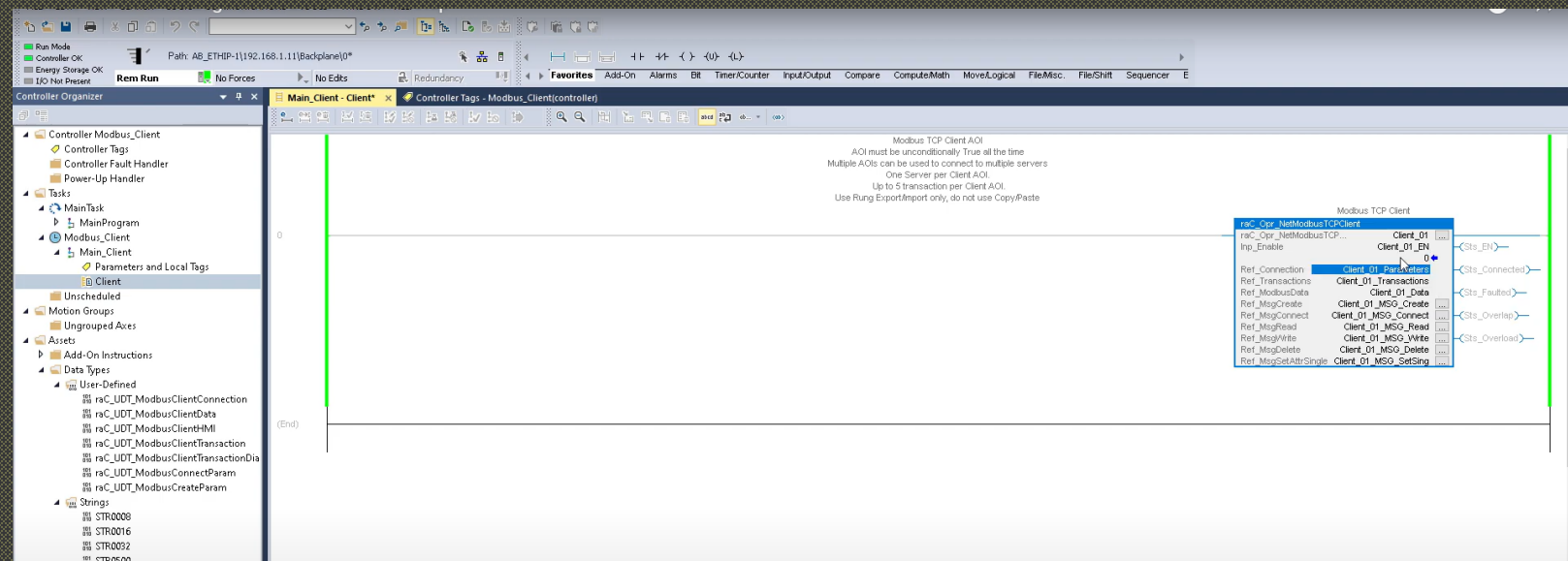


7A.7 Modbus in Siemens and Allen-Bradley PLCs

The ControlLogix platform from Rockwell Automation also supports Modbus TCP communication, but it is not natively integrated in the firmware like in Siemens PLCs.

- The PLC can function as a Modbus TCP client (master) to actively poll data from Modbus servers, or as a Modbus TCP server (slave) to make its own data available to other Modbus clients.
- This functionality is typically achieved using either:
 - A Rockwell Add-On Instruction (AOI) that implements the Modbus TCP protocol in ladder logic, or
 - A dedicated communication module (such as the 1756-EN2TR or third-party Modbus TCP module) that manages the protocol conversion.

The AOI or communication module maps Modbus registers to ControlLogix tags, enabling seamless data exchange between Modbus-based systems and the PLC's internal memory.



REFERENCES

- MODBUS APPLICATION PROTOCOL SPECIFICATION V1.1b3
- Modbus: The Everyman's Guide to Modbus, John S. Rinaldi

INDUSTRIAL NETWORKING

MENG 4010

Thank you!

Discussions?